

Mining Point-of-No-Return Boundaries in Constrained Dynamical Systems via Counterfactual Auditing

Anonymous Author(s)

Abstract

Safety-critical failures in constrained dynamical environments are often detected only after a violation occurs, while outcome metrics (e.g., success rate, time-to-failure) conflate structural inevitability with decision-induced errors. We formulate failure diagnosis as recoverability boundary discovery and define the Point-of-No-Return (PoNR) as the earliest snapshot after which violation becomes unavoidable under an audited intervention class. We introduce Counterfactual PoNR Auditing (CPA), a snapshot-replay framework that produces matched counterfactual rollouts and is accelerated by a neural operator surrogate (UNO) for a $3.09\times$ end-to-end speedup. CPA yields Evidence Objects (EOs) that support structural-policy decomposition and enable Failure Atlas mining via contrastive analysis of event sequences within the policy-induced gap. CPA also serves as an offline supervision generator, allowing lightweight predictors to learn early inevitability signals beyond static observational baselines. Experiments on a high-fidelity hydrodynamic control testbed and a canonical bistable system show that PoNR precedes observable violation, remains stable under up to 30% multiplicative surrogate noise, and reduces auditing cost through surrogate-guided search.

CCS Concepts

• **Information systems** → **Data mining**; • **Computing methodologies** → *Artificial intelligence*; Causal reasoning and diagnostics; Modeling and simulation.

Keywords

Constrained Dynamical Systems, Point-of-No-Return (PoNR), Counterfactual Auditing, Recoverability Boundaries, Evidence Objects, Contrastive Sequential Pattern Mining, Failure Motifs

ACM Reference Format:

Anonymous Author(s). . Mining Point-of-No-Return Boundaries in Constrained Dynamical Systems via Counterfactual Auditing. In . ACM, New York, NY, USA, 19 pages.

1 Introduction

Autonomous decision-making systems are increasingly deployed in high-stakes constrained dynamical environments, ranging from industrial safety control to infrastructure monitoring and emergency response [15, 17, 26]. Across classical controllers, reinforcement learning policies, and tool-using agents, failures in constrained dynamics can exhibit phase-like transitions [18, 20, 24]. An episode may remain feasible and appear stable in observation, yet already be irrecoverable under stochastic disturbances. This decoupling creates a Zombie Zone (Fig. 1): a feasible-but-irrecoverable window in which sensors report no unsafe indications even though the system has crossed a recoverability boundary.

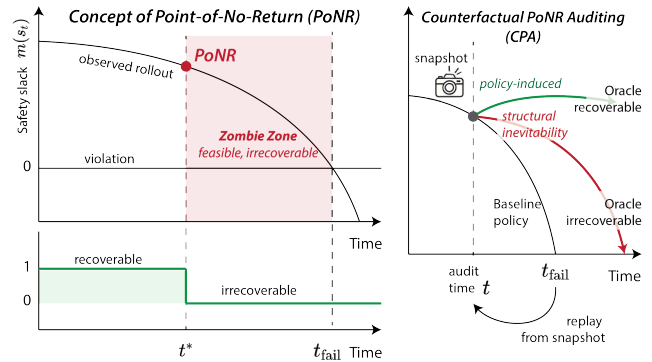


Figure 1: Point-of-No-Return (PoNR) and counterfactual auditing. Left: PoNR t^* is the last recoverable time; the interval $[t^*, t_{\text{fail}}]$ is feasible but irrecoverable (Zombie Zone). Right: CPA replays a snapshot at time t and branches rollouts to separate policy-induced failure (oracle recoverable) from structural inevitability (oracle irrecoverable).

Outcome metrics such as success rate and time-to-failure are typically lagging indicators that conflate two distinct sources of failure. The first is structural irrecoverability induced by environment dynamics and disturbances [6, 13]. The second is policy-induced irrecoverability, where recovery was structurally feasible but became unattainable due to the decision-maker's choices [5, 9]. This distinction is critical in agentic evaluations, where outcomes are often highly sensitive to randomness and implementation details [10]. Unlike per-timestep failure prediction, which treats each step as an i.i.d. classification instance [12], diagnosing irrecoverability is a boundary discovery problem over sequential trajectories. This perspective aligns naturally with rare-event analysis and contrastive pattern mining [7].

To make irrecoverability explicit, we propose Point-of-No-Return (PoNR) boundary mining. PoNR identifies the last snapshot where an episode remains recoverable under an audited intervention class; beyond this point, eventual constraint violation becomes unavoidable. Notably, this boundary can precede observable failure: constraints may still be satisfied, yet the remaining slack is insufficient to enable recovery. As an offline supervision label produced by counterfactual auditing, PoNR can be distilled into learnable early-warning monitors. We operationalize this via Counterfactual PoNR Auditing (CPA), which leverages replayable simulator snapshots to branch controlled counterfactual rollouts. CPA yields Evidence Objects (EOs): snapshot-aligned counterfactual pairs that support structural-policy attribution and reduce sensitivity to environmental randomness through paired replay.

Building on EOs, we introduce Failure Atlas Mining to extract reusable failure motifs. Inspired by counterfactual contrastive analysis [4, 16], we analyze event sequences within the policy-induced gap and identify an Oscillation Trap motif via snapshot-matched

replay. To address the computational bottleneck of counterfactual auditing, we integrate a U-Net Operator (UNO) surrogate, reducing the marginal cost of counterfactual rollouts by orders of magnitude (8ms vs. 72min) with negligible accuracy loss (NSE > 0.99; Appendix F). We evaluate the framework on a high-fidelity hydrodynamic control testbed and a canonical bistable dynamical system, and demonstrate stability under up to 30% surrogate noise (Appendix F).

Our contributions are as follows.

- We formalize PoNR as a recoverability boundary and introduce CPA to produce snapshot-aligned EOs, enabling counterfactual attribution of structural inevitability versus decision-induced errors.
- We use CPA as an offline label-generation engine for downstream tasks, including **training** lightweight early-warning monitors (AUC > 0.95) and contrastively **mining** interpretable failure motifs (Failure Atlas).
- We validate CPA on a canonical bistable dynamical system, showing that the Zombie Zone can arise generically in constrained dynamics beyond a specific control testbed.

2 Related Work

2.1 From outcome metrics to boundary auditing

Evaluation in high-stakes constrained dynamical environments is commonly outcome-centric, reporting success rate, constraint violations, and cost, often combined with runtime guardrails that treat failure as a terminal event [1, 25, 28]. In parallel, safe reinforcement learning and constrained control prioritize prevention, either by optimizing CMDP-style objectives or enforcing invariance-based constraints during training and execution [8, 11]. A complementary line of work focuses on prediction: early-warning models and survival-style analyses estimate statistical risk to forecast if and when failure will occur [21]. Our work targets a diagnostic objective that is orthogonal to both prevention and prediction: we audit a deployed decision policy—which may be a classical controller, an RL policy, or an agentic system—to localize the time at which recoverability is lost under feasible interventions, i.e., the point-of-no-return [2, 27]. This perspective reframes failure analysis from outcome summaries or risk scoring to discovering a dynamical recoverability boundary embedded in closed-loop interactions.

2.2 Counterfactual alignment for sequential mining and scalable auditing

Counterfactual explanation and auditing methods are most often developed for static prediction settings, where counterfactuals modify inputs to change a model output [3, 14, 19]. In contrast, CPA operates in sequential closed-loop systems by branching rollouts from replayable snapshots. This snapshot-replay protocol produces aligned Evidence Objects that support matched comparisons and help separate decision-induced vulnerability from structural inevitability. Building on these aligned traces, our Failure Atlas performs contrastive sequential pattern mining, differing from observational log mining where correlations can confound attribution [22, 23]. To mitigate the cost of boundary auditing in expensive simulators, we

leverage surrogate guidance to prioritize boundary-relevant queries and report the resulting efficiency behavior in the experiments.

3 Problem Formulation

We study counterfactual auditing of safety-critical risk episodes in constrained dynamical environments. Our objective is not to optimize control performance, but to identify the recoverability boundary—the latest time at which an intervention can still prevent future constraint violation.

3.1 Episodes and replayable snapshots

We consider a risk episode recorded as a discrete-time snapshot sequence $\{x_t\}_{t=0}^T$, where each snapshot x_t is replayable under a fixed simulator and disturbance model. Each snapshot contains the information required to restart simulation from time t , enabling counterfactual rollouts from the same starting point. Let s_t denote the underlying physical state contained in x_t .

Safety is defined by a slack function $m : \mathcal{S} \rightarrow \mathbb{R}$:

$$\begin{aligned} m(s_t) \geq 0 & \text{ indicates feasibility,} \\ m(s_t) < 0 & \text{ indicates violation.} \end{aligned} \quad (1)$$

In our hydrodynamic testbed, $m(s_t)$ is instantiated as the safety margin to the dyke/limit water level (i.e., the remaining clearance before overtopping).

We define the observed failure time for an episode as

$$t_{\text{fail}} \triangleq \min\{t \in \{0, \dots, T\} \mid m(s_t) < 0\}, \quad (2)$$

if a violation occurs within the observation window.

3.2 Recoverability and point of no return

We define the intervention space as a functional space $\mathcal{U}$. In our hydrodynamic setting, the action at time t is the normalized drainage intensity $u_t \in [0, D_{\text{max}}]$. CPA evaluates interventions as do-operators applied at a snapshot x_t . Let ι denote a fixed audited recovery capability. Specifically, we adopt the maximal bang-bang control $\iota_{\text{oracle}} = \{u(t) \equiv D_{\text{max}}\}$ as the reference protocol. In this dissipative system, the water level is monotonically non-increasing with respect to drainage intensity; thus, ι_{oracle} provably yields the optimal recovery trajectory without requiring complex MPC search.

Given a horizon H and tolerance $\eta \in [0, 1)$, we define future-only (η, H) -recoverability:

$$\begin{aligned} y(x_t; \iota) &= \mathbb{I}\left[\forall k \in [t+1, t+H], \right. \\ &\quad \left. m(s_k) \geq 0 \mid x_t, \text{do}(\iota) \right] \geq 1 - \eta \end{aligned} \quad (3)$$

where $\mathbb{I}[\cdot]$ is the indicator function. While formulated probabilistically to accommodate surrogate uncertainty, for deterministic simulator auditing (as in our primary experiments), the probability collapses to a binary check (i.e., we set $\eta = 0$).

Along a risk episode, $y(x_t; \iota)$ typically transitions from 1 (still savable) to 0 (already lost). We define the Point-of-No-Return as the latest savable snapshot time:

$$t^*(\iota) \triangleq \max\{t \in \{0, \dots, T\} \mid y(x_t; \iota) = 1\}. \quad (4)$$

In all experiments, the reported t_{PoNR} is the estimate $\hat{t}^*(\iota)$ returned by CPA (Sec. 4.2).

The Actionable Lead Time is the temporal gap between PoNR and the observed violation:

$$\Delta \triangleq \max(0, t_{\text{fail}} - \hat{t}^*(\iota)). \quad (5)$$

When $\Delta > 0$, the interval $[\hat{t}^*(\iota), t_{\text{fail}}]$ is the *Zombie Zone*, a phase where the system remains feasible yet has already lost recoverability under ι .

3.3 Structural and policy decomposition

To separate system limits from decision quality, we compare PoNR under two audited capability settings: a deployed setting ι_{obs} and an upper-bound setting ι_{oracle} that represents enhanced recovery while keeping the same environment dynamics fixed. Let $\hat{t}_{\text{obs}}^*$ and $\hat{t}_{\text{oracle}}^*$ denote the CPA estimates under these two settings.

We define the policy-induced gap as

$$\Delta_{\text{gap}} \triangleq \hat{t}_{\text{oracle}}^* - \hat{t}_{\text{obs}}^* \geq 0. \quad (6)$$

Intuitively, Δ_{gap} measures how much earlier the episode becomes irrecoverable under the deployed setting compared to the upper bound, thus localizing the segment where improved decision-making could have extended viability.

4 Methodology

Having formalized PoNR and its decomposition, we now introduce **Counterfactual PoNR Auditing (CPA)**, a framework to estimate these boundaries and mine failure patterns. CPA operates on *snapshot-based time-travel*: it forks simulation from a recorded snapshot x_t to explore counterfactual futures under controlled interventions. Throughout, we assume replayable snapshots $x_t = (s_t, h_t, b_t, r_t)$ as defined in Section 3.

4.1 Evidence Objects: The Mining Unit

To enable reproducible auditing and downstream mining, we define an **Evidence Object (EO)** as a compact, replayable record of a snapshot-based counterfactual test:

$$E \triangleq \langle x_{\text{start}}, \iota, \tau', \mathbf{m}', \mathbf{u}', y, t_{\text{fail}}, c, \text{meta} \rangle, \quad (7)$$

where x_{start} is the branching snapshot (at step t_{start}), and $\iota \in \mathcal{I}$ specifies the intervention (e.g., switching to an oracle recovery policy). The counterfactual rollout τ' records the resulting trajectory; recoverability is evaluated on an H -step prefix, with slack sequence $\mathbf{m}' = \{m(s_k)\}_{k=t_{\text{start}}}^{t_{\text{start}}+H}$. $\mathbf{u}'$ denotes the tool-call/event stream, $y \in \{0, 1\}$ is the (ir)recoverability outcome, t_{fail} is the first violation time if it occurs, and c summarizes auditing cost. Finally, meta stores reproducibility identifiers (domain, simulator version, policy ID, seed).

Unlike raw logs, EOs are *snapshot-aligned* across interventions (sharing the same x_{start}) and therefore comparable instances for matched counterfactual analysis. We aggregate EOs into a dataset $\mathcal{D}$, which serves as the input to both causal auditing and Failure Atlas mining.

4.2 Algorithm: Counterfactual PoNR Auditing (CPA)

CPA estimates the *Point-of-No-Return* as a **transition time** on the snapshot timeline of a risk episode. Concretely, for each auditable

episode (Track A; Sec. 5.1), we consider the discrete snapshot sequence $\{x_t\}_{t=0}^T$ (1-hour resolution in our setting) and a fixed audited recovery capability ι (e.g., max-drain oracle). CPA searches for the latest snapshot time t^* such that starting the intervention at x_{t^*} still yields a feasible future for a horizon H . The resulting estimate $\hat{t}^*$ is reported as t_{PoNR} in all experimental figures and tables (e.g., Fig. 3, Fig. 6).

Recoverability as a binary transition. The future-only (η, H) -recoverability indicator $y(x_t; \iota)$ is defined as in Equation (3). In inertia-dominated systems, $y(x_t; \iota)$ typically exhibits a one-way transition from 1 (savable) to 0 (lost) along the episode timeline. CPA therefore estimates PoNR by locating the **boundary index** of this transition.

Track taxonomy and censoring (aligned with experiments). We integrate the experimental episode taxonomy directly into the audit protocol:

- **Track B (Instant Fail):** episodes with observed failure within two snapshots ($t_{\text{fail}} < 2$ hours) are excluded from boundary search and reported separately (Sec. 5.1).
- **Left-censored:** if $y(x_{t_0}; \iota) = 0$ at the earliest auditable time t_0 (episode start), the episode is unsavable under the audited capability class; we record $\hat{t}^*$ as left-censored.
- **Right-censored:** if $y(x_{t_{\text{max}}}; \iota) = 1$ at the latest snapshot in the observation window, PoNR lies beyond the window; we record $\hat{t}^*$ as right-censored.
- **Observed:** otherwise, PoNR is identifiable within-window and we return a finite $\hat{t}^*$ used in lead-time statistics.

Transition localization over snapshots. We localize the recoverable-to-irrecoverable transition by bisection over snapshot indices. A lightweight surrogate may propose a near-boundary bracket $[t_a, t_b]$, but the final decision is always verified by high-fidelity counterfactual rollouts from the candidate snapshot. Each oracle query produces an Evidence Object (Sec. 4.1), so the audit outputs both a PoNR estimate and a replayable evidence dataset. The full procedure is deferred to Appendix C.3.

Output semantics (hard alignment to experiments). For each non-censored Track A episode, CPA outputs $\hat{t}^*$, which is the t_{PoNR} used in Fig. 3 and to compute lead time Δ . Episodes are categorized by Δ into *Structural* ($< 0.5\text{h}$), *Policy Gap* (intermediate), and *Manageable* ($> 2.5\text{h}$), matching the composition plot in Fig. 5. Censoring flags determine inclusion in lead-time statistics and regime composition.

4.3 Causal Estimands

CPA produces snapshot-aligned Evidence Objects (Sec. 4.1) by branching multiple do-operator interventions from the same replayable snapshot x_t . This alignment enables paired causal comparisons under identical starting conditions, so that differences are attributable to interventions rather than episode selection.

PoNR boundary and recoverability. For an intervention $\iota \in \mathcal{I}$, CPA returns (i) a PoNR estimate $\hat{t}^*(\iota)$, defined as the last recoverable snapshot index under the (η, H) test used by Algorithm 1, and (ii) a recoverability outcome $y(\iota) \in \{0, 1\}$ evaluated on the same auditable snapshot set. In the experiments, $\hat{t}^*(\iota)$ corresponds to the PoNR time reported in phase-diagram figures (e.g., t_{PoNR}), while

$y(i)$ corresponds to whether the episode is recoverable under i given the auditing horizon.

Paired average treatment effects. Let i_0 denote the baseline capability (e.g., the deployed policy class). We focus on two estimands that directly map to the experimental evidence chain:

$$\text{ATE}_{t^*}(i) \triangleq \mathbb{E}[\hat{t}^*(i) - \hat{t}^*(i_0)], \quad (8)$$

$$\text{ATE}_{\text{RSR}}(i) \triangleq \mathbb{E}[y(i) - y(i_0)]. \quad (9)$$

Expectations are taken over audited episodes and simulator randomness; differences are computed on snapshot-matched EO groups produced by counterfactual parallelism. ATE_{t^*} measures how an intervention shifts the recoverability boundary (earlier vs. later PoNR), while ATE_{RSR} measures the corresponding change in recoverability probability.

Policy-induced gap and actionable lead time. A central quantity in our decomposition is the *policy-induced gap*, Δ_{gap} , which was defined in Eq. (6) as the expected difference between the PoNR under an oracle (i_{oracle}) and the deployed (i_{obs}) capability classes. A larger Δ_{gap} indicates that failures are, to a greater extent, attributable to decision quality (recoverability lost earlier under the deployed class) rather than structural inevitability. Correspondingly, the *actionable lead time*, Δ , was defined in Eq. (5) and quantifies the window between the structural PoNR (under i_{oracle}) and the observed violation. This is the operational window that is invisible to reactive thresholding yet detectable by recoverability auditing.

Cost reporting. CPA admits a natural cost notion through the number of oracle calls induced by boundary search. When detailed runtime is not the focus, we summarize cost primarily by oracle-call counts and the logarithmic bisection behavior implied by Algorithm 1, and emphasize boundary/lead-time effects as the primary evidence.

4.4 Identification and Robustness

CPA targets counterfactual estimands that are identified by design in a replay-based setting. The central device is *counterfactual parallelism*: multiple interventions branch from the same replayable snapshot x_t , yielding aligned EOs. Accordingly, all paired effects in Sec. 4.3 are interpreted *on the auditable population* and under the oracle simulator.

Auditable population and episode filtering (Track A/B). CPA is executed only on episodes for which replayable snapshots exist and oracle rollouts can be completed. We denote this auditable subset as **Track A**. Episodes that cannot be audited due to non-replayable conditions (e.g., instant failure at the first snapshot, missing replay state, or other execution failures) are categorized as **Track B** and excluded from the EO dataset used for effect estimation. All estimands in Sec. 4.3, including ATE_{t^*} , ATE_{RSR} , and Δ_{gap} , are therefore defined with respect to Track A.

Censoring and well-defined EO outcomes. For each snapshot x_t and intervention i , the recoverability label $y(x_t; i)$ is defined via the finite-horizon (η, H) test using K oracle rollouts. If oracle verification cannot be completed for a branch (e.g., truncated rollouts or missing simulator outputs), the corresponding EO is treated

as *censored* and excluded from snapshot-matched paired comparisons. This convention ensures that both $\hat{t}^*(i)$ and $y(x_t; i)$ entering Sec. 4.3 are computed only from fully evaluated counterfactual tests, avoiding ambiguous outcomes.

Identification assumptions. Within Track A, identification follows from standard assumptions that are operationally enforced by the snapshot-replay protocol. First, *consistency* holds because counterfactual outcomes are generated by running the oracle simulator from x_t under the specified do-operator $do(i)$. Second, *exchangeability* is ensured by construction: interventions share the same starting snapshot, and randomness arises only from independent disturbance draws (and agent sampling noise, when applicable) across oracle rollouts. Third, *no interference* holds because counterfactual branches are executed independently across episodes.

Robustness and Model Fidelity. We complement the design-based identification with rigorous stress tests to verify that the mined boundary is not an artifact of surrogate approximation. In particular, we examine the stability of $\hat{t}^*$ under surrogate model uncertainty by injecting multiplicative Gaussian noise (up to $\sigma = 30\%$) into the rollout dynamics (detailed in Appendix F.3). The estimated PoNR boundary shifts by less than ± 1 hour under moderate noise, confirming that the Zombie Zone is a stable structural feature robust to high-frequency model errors. Additionally, we verify causal effects using within-group label permutation to ensure that paired differences concentrate near zero under randomized interventions. Together, these properties justify interpreting ATE_{t^*} as a structural shift in recoverability, robust to both simulator imperfections and procedural choices.

5 Experiments

We evaluate CPA on a high-fidelity hydrodynamic control testbed of storm-surge dynamics in the Yangtze Estuary. This domain exemplifies **inertia-dominated** constrained systems: safety violations are rare but catastrophic, and interventions exhibit delayed consequences, making purely observational alarms prone to lagging behavior.

Our goal is to challenge a common monitoring paradigm in such systems: instead of treating safety as a property of the *current state*, we test whether *recoverability*—a dynamic property assessed via counterfactual futures—provides a more reliable **leading signal** of impending constraint violation. Beyond early warning, we ask when such signals are operationally useful: can CPA separate *decision-sensitive* failures (where improved actions could still matter) from *structural* inevitability under a fixed audited recovery capability?

The experiments follow a tightly coupled evidence chain:

H1 (Existence): Zombie Zone. There exist episodes that remain observationally feasible yet have already become irrecoverable under the audited capability class. We establish this by case-level evidence (Fig. 2).

H2 (Leading signal & baseline failure): PoNR precedes violation. The structural boundary mined by CPA statistically precedes observed violation, yielding an actionable window. We validate this via population-level phase diagrams (Fig. 3) and show that

static threshold alarms can be systematically late across sensitivity settings (Table 2).

H3 (Operational regimes): value depends on capacity. We test whether increasing recovery capacity is associated with a shift from structurally inevitable failures toward decision-sensitive regimes where auditing provides practical debugging value (Fig. 6).

Finally, we provide a mechanistic explanation for why static alarms can fail: early-stage trajectories can be deceptively similar across regimes, while short-horizon dynamics (momentum features) enable accurate early warning (Fig. 4).

5.1 Auditing Protocol & Experimental Setup

We study replayable hydrodynamic control in the Yangtze Estuary under fixed exogenous forcing. The underlying dynamics are defined by a high-fidelity simulator (Delft3D). To accelerate counterfactual boundary search, we use a learned neural operator (UNO) and reserve oracle simulator runs for final verification (Sec. 5.4).

We construct a corpus of risk episodes with snapshots recorded at $\Delta t = 1\text{h}$ and partition them by temporal resolution. Track A contains auditable episodes with sufficient observation windows ($t_{\text{fail}} \geq 2\Delta t$) to localize a boundary via bracketing; Track B contains instant failures ($t_{\text{fail}} < 2\Delta t$) and is excluded from boundary mining because binary search cannot be bracketed. This is distinct from left-censored episodes in Track A, which are valid long episodes but structurally unrecoverable even at $t = 0$. Dataset statistics are reported in Appendix A.

The action space is normalized drainage intensity $\mathcal{A} = \{a_t \in [0, D_{\text{max}}]\}$. We instantiate CPA with a single maximal-intensity intervention t_{max} where $a_t \equiv D_{\text{max}}$, treating the intervention class ι as a fixed policy. In this dissipative setting, we adopt a protocol-level monotonicity assumption: within the audited class, larger drainage does not decrease the safety margin over the fixed horizon, so t_{max} serves as an upper bound on recoverability and avoids MPC-style search when defining the boundary.

While Eq. 3 defines a general stochastic predicate, our primary audit exploits deterministic snapshot replay. We therefore instantiate recoverability with $K=1$ and $\eta=0$ (up to a small numerical tolerance), and reserve probabilistic auditing ($K>1$) for surrogate-uncertainty analysis under noise injection (Appendix F.3, Fig. 17). For each valid episode, PoNR t^* is the latest snapshot such that initiating t_{max} keeps the system feasible over a fixed horizon $H=24$ hours.

We report baseline failure time t_{fail} , PoNR time t^* , and actionable lead time $\Delta = \max(0, t_{\text{fail}} - t^*)$ in absolute time units, and summarize population-level actionable rate $\mathbb{P}[\Delta \geq 1\text{ hour}]$. All auditing logs are released to enable deterministic replay.

5.2 Zombie Zone: Existence and Precedence

5.2.1 Micro-Dynamics (H1). We first provide micro-level evidence for the proposed *Zombie Zone*: a critical interval in which the system remains observationally feasible (positive safety margin) yet has already become irrecoverable under the audited capability class.

Figure 2 visualizes a representative failure episode, revealing a systematic lag between *observed feasibility* and *counterfactual recoverability*. Although the safety margin (slack) remains positive for several hours, the recoverability signal drops instantly at t_{PoNR}

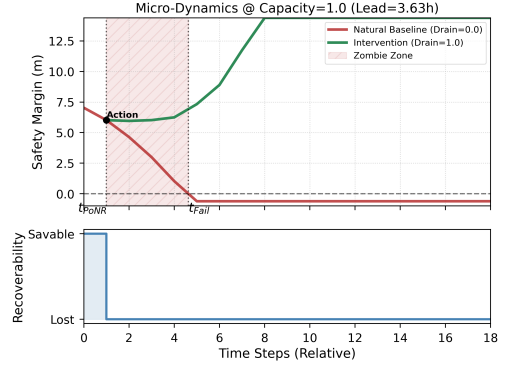


Figure 2: A representative failure episode illustrating the Zombie Zone. Top: Safety margin under baseline (red) and oracle intervention (green). Bottom: Recoverability indicator transition from Savable to Lost. Shaded region = Zombie Zone [t_{PoNR} , t_{fail}].

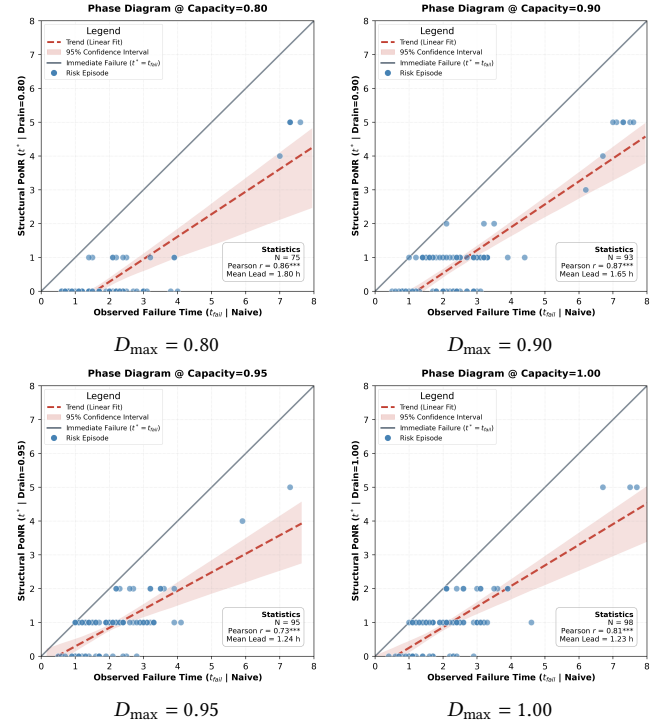


Figure 3: Phase diagrams across audited intervention capacities. Each point is a replayable risk episode, plotting the structural boundary time t_{PoNR} against the observed failure time t_{fail} . The gray diagonal indicates zero lead time ($t_{\text{PoNR}} = t_{\text{fail}}$).

(denoted as t^* in the audit). Crucially, even the maximum-strength intervention (green trajectory), despite visibly deflecting the slack trajectory, fails to avert the violation at t_{fail} . This decoupling is characteristic of inertia-dominated systems: exogenous forcing and accumulated momentum can render failure inevitable well before the state variable crosses the safety threshold.

Table 1: Sensitivity of actionable lead time Δ to threshold ratios (risk episodes at $D_{\max} = 1.0$).

Threshold Ratio	N	Pearson r	Mean Δ (h)	Median Δ (h)	$\mathbb{P}[\Delta \geq 1\text{h}]$
0.80	75	0.86	1.80 ± 0.81	1.70 (IQR 1.15)	0.84
0.85	82	0.85	1.54 ± 0.76	1.50 (IQR 1.30)	0.73
0.90	93	0.87	1.65 ± 0.83	1.90 (IQR 1.40)	0.73
0.95	95	0.73	1.24 ± 0.73	1.20 (IQR 1.10)	0.65
1.00	98	0.81	1.23 ± 0.74	1.10 (IQR 1.15)	0.64

Operationally, this confirms that threshold-based monitoring on slack is a lagging indicator. The interval gap Δ constitutes the *Actionable Lead Time* exposed by our recoverability audit. This micro-level anatomy motivates the population-level precedence analyses in Section 5.2.2. Additional case studies are provided in Appendix E.1.

5.2.2 Population-Level Precedence (H2). We extend the analysis from micro-dynamics to the population level by quantifying the temporal precedence of the structural boundary (t_{PoNR}) relative to the observational failure (t_{fail}) across all valid risk episodes.

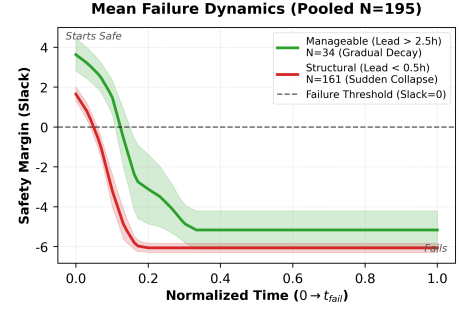
Precedence and Actionability. Figure 3 plots t_{PoNR} against t_{fail} under multiple audited intervention capacities. Our audit reveals that in many episodes the system enters the irrecoverable regime earlier ($t_{\text{PoNR}} < t_{\text{fail}}$), yielding positive lead time Δ . This separation confirms hydro-dynamical inertia creates a "blind spot" for instantaneous constraints, which our look-ahead mechanism successfully exposes. Quantitatively, the induced lead time is consistently non-zero: across threshold ratios, the median Δ remains around 1–2 hours with a non-trivial actionable rate (Table 1).

Robustness to Safety Thresholds. To assess whether the leading-signal effect depends on a particular slack threshold, we fix the audited capability at the maximum oracle ($D_{\max} = 1.0$) and vary the violation ratio from 0.80 to 1.00 relative to historical peaks, isolating thresholding from capacity effects. Table 1 reports the resulting lead-time and association statistics.

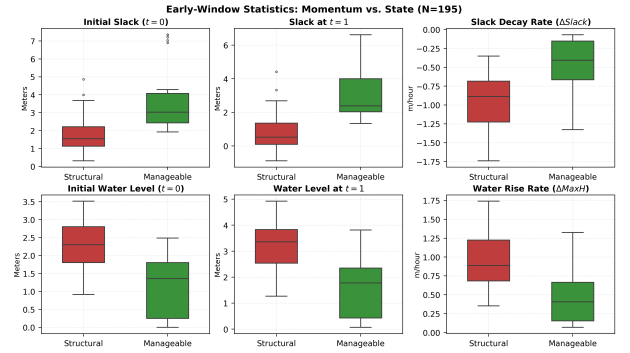
Across all ratios, the lead time Δ exhibits a consistently positive central tendency with a non-trivial actionable rate, indicating that the PoNR boundary is not a threshold-tuned artifact. While stricter thresholds naturally trigger earlier declarations and hence larger leads, the effect persists even at the peak-referenced setting (ratio = 1.00), where the mean lead time remains above 1 hour.

Table 2: Comparison of Late-Alarm Rate and Alarm Lag between CPA and static threshold baselines across intervention capacities.

Capacity ($D_{\max}$)	Aggressive ($\tau = 0.5\text{m}$)		Conservative ($\tau = 1.0\text{m}$)	
	Late Rate	Lag (h)	Late Rate	Lag (h)
0.80	98.7%	2.01	81.3%	1.72
0.85	96.3%	1.77	73.2%	1.70
0.90	95.7%	1.90	73.1%	1.71
0.95	85.3%	1.59	63.2%	1.42
1.00	83.7%	1.60	58.2%	1.44



(a) Mean slack dynamics: early overlap.



(b) Early-window stats: static vs. momentum features.

Figure 4: Deceptive early states and momentum signals. (a) Structural and Manageable episodes exhibit substantial early overlap in slack trajectories, causing static thresholds to trigger late alarms. (b) Momentum features (e.g., ΔSlack , ΔMaxH) provide stronger early separability than static state features.

5.2.3 Comparative Analysis: Failure of Static Baselines. We benchmark CPA against standard observational baselines to validate the necessity of counterfactual auditing. We evaluate a threshold-based detector that triggers an alarm at $t_{\text{alarm}}(\tau) = \min\{t \mid \text{Slack}_t < \tau\}$.

Table 2 reveals a fundamental limitation of observational monitoring. The results demonstrate that static constraints act as lagging indicators of irrecoverability. Even with conservative threshold ($\tau = 1.0\text{m}$), the static alarm lags behind the structural boundary in 58.2% of episodes at maximum capacity. Across capacities, CPA provides an average of 1.4–1.6 hours of additional reaction time relative to static alarms.

5.2.4 Deceptive Early States and Momentum Signals. Static baselines fail because early states are often *deceptive*: manageable and irrecoverable episodes can exhibit similar slack trajectories before the recoverability transition. Figure 4(a) shows substantial early overlap between Structural and Manageable groups, explaining why fixed thresholds incur late alarms. However, short-horizon *momentum* signals (e.g., ΔSlack , ΔMaxH) are more separable than instantaneous state values (Fig. 4(b)), suggesting that irrecoverability is encoded in early acceleration patterns rather than slack alone.

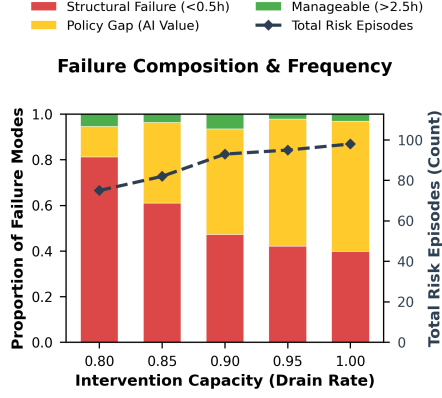


Figure 5: Failure composition vs. capacity. Stacked bars show regime proportions by lead time Δ : Structural ($\Delta < 0.5h$), Policy Gap (intermediate), and Manageable ($\Delta > 2.5h$). Dashed line (right axis): number of risk episodes.

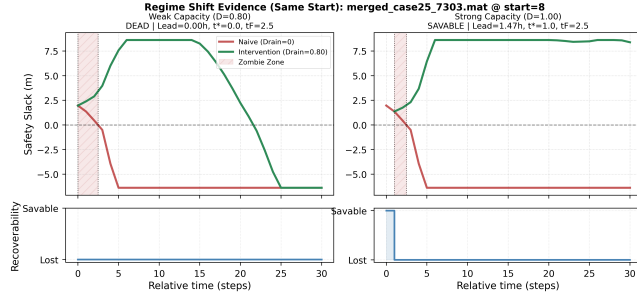


Figure 6: Case Study: Capacity-Dependent Regime Shift. Same snapshot ($t_{\text{start}} = 8$) under two limits: $D_{\text{max}} = 0.80$ (left) remains Structural; $D_{\text{max}} = 1.00$ (right) becomes recoverable under the oracle. Only D_{max} changes.

5.3 Operational Utility: Regime Taxonomy, Early Warning, and Motif Mining

5.3.1 Operational Regimes (H3). We analyze how failure composition changes with intervention capacity to locate where CPA provides the highest operational value. Figure 5 shows a clear regime shift: at low capacity, most risk episodes are structural (near-zero lead time, $< 0.5h$), indicating physically inevitable failures. As capacity increases, the mass moves into the Policy Gap region, where recovery is feasible in principle but missed by the baseline policy. A small fraction becomes clearly manageable (large lead time, $> 2.5h$), reflecting substantial slack. This trend implies that higher capacity does not eliminate risk; it converts inevitability into policy-sensitive, windowed failures—precisely the setting where snapshot-aligned counterfactual auditing is most informative. Figure 6 visualizes the same transition on a single episode under identical snapshots.

5.3.2 Learnability of PoNR Labels. CPA is an offline audit, not an online predictor. Its role is to produce supervision: PoNR labels and aligned Evidence Objects that can be learned by lightweight models. Using only the first two snapshots (2h window), a simple Random Forest classifier predicts whether $t_{\text{PoNR}} \leq 1.5h$ with high AUC (Fig. 7). An ablation without the capacity feature remains

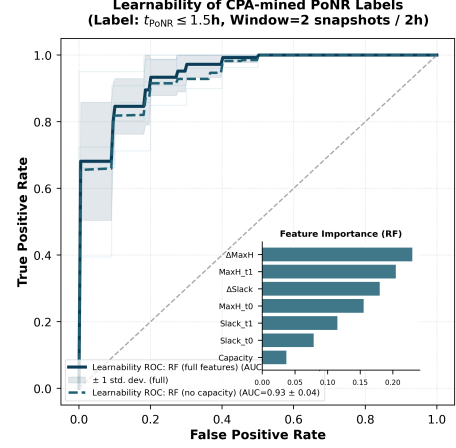


Figure 7: Learnability of CPA-mined PoNR labels. A Random Forest trained on two early snapshots achieves strong ROC-AUC for predicting $t_{\text{PoNR}} \leq 1.5h$, and remains competitive without capacity as an input feature. This suggests PoNR provides a learnable supervision signal rather than a hand-crafted threshold.

strong, indicating that learnability is not merely driven by control authority but also by short-horizon dynamics.

5.3.3 Failure Atlas Excerpt. To substantiate the mining capability enabled by EOs, we highlight an intuitive Zombie-Zone failure motif, the Oscillation Trap, where control degenerates into high-frequency bang-bang adjustments under shrinking safety margins. We run a snapshot-aligned counterfactual replay on $N=30$ identified risk episodes: from the same branching snapshot and under identical forcing, we replay (i) a Panic controller that induces rapid action fluctuation and (ii) a Smooth reference controller. Control instability is quantified by fluctuation intensity $|\Delta u|_1$.

Figure 8 shows a clear distributional separation: Panic exhibits higher oscillation (Mean=0.430) than the Smooth reference (Mean=0.265) on the same snapshot set, indicating that aligned EOs support contrastive motif extraction attributable to behavioral differences rather than episode selection.

Beyond the scalar metric, the aligned EO pairs enable *contrastive sequential pattern mining*. We tokenize each rollout using a physics-informed discretization operator (Appendix E.4), mapping the normalized slack ratio $r_t = (\theta - h_t)/\theta$ to safety contexts (Safe/Warn/Near/Crit) and mapping action changes $\Delta u_t = u_t - u_{t-1}$ to dynamics symbols (Up++/Up/Dn++/Dn/St), forming tokens such as Warn.St. We then mine N -gram motifs enriched in the failure-inducing branch (Panic) relative to the reference (Smooth) under identical episode conditions. The full procedure (episode-level support, one-sided Fisher test, and Benjamini-Hochberg FDR control) is summarized in Algorithm 2 and detailed in Appendix E.4.

5.4 Scalability and Generalization

5.4.1 Efficiency via Surrogates. Boundary mining in high-fidelity dynamical systems is bottlenecked by expensive simulator rollouts. We benchmark CPA on 22 long-horizon episodes (audit horizon > 3 steps, 1-hour resolution) using measured latencies: **72.0 min** per oracle rollout (Delft3D) versus **8.42 ms** per surrogate step

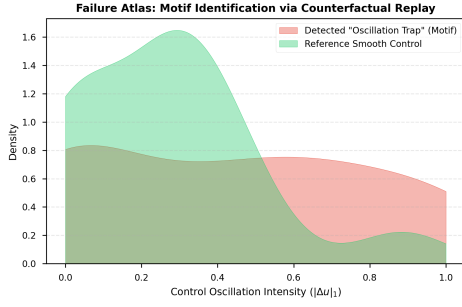


Figure 8: Failure Atlas motif discovery via snapshot-aligned counterfactual replay. Two controllers are replayed from identical branching snapshots across $N=30$ risk episodes and evaluated by $|\Delta u|_1$. Panic exhibits higher oscillation (Mean=0.430) than Smooth (Mean=0.265), yielding a clear separation.

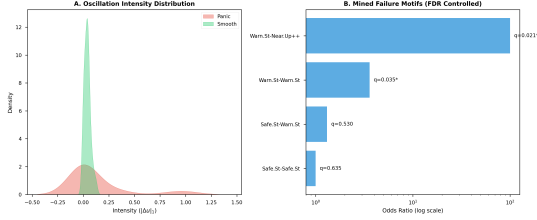


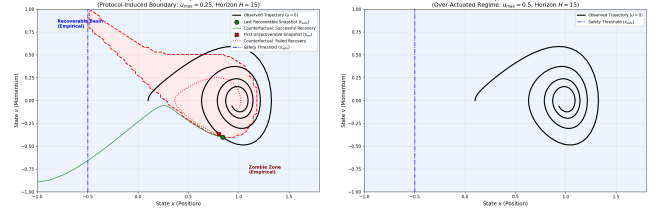
Figure 9: Failure Atlas Mining via snapshot-aligned Evidence Objects. (A) Micro-dynamics view: Panic exhibits higher oscillation intensity than Smooth under identical starting snapshots. (B) Mining view: contrastive 2-gram motifs mined via one-sided Fisher exact test with Benjamini–Hochberg FDR control (Algorithm 2; Appendix E.4). Odds ratios are log-scaled; infinite values are capped for visualization.

(UNO). Compared to a standard binary-search baseline (3.09 oracle queries/episode), surrogate-guided localization reduces verification to 1.00 oracle query/episode (67.6% fewer queries), yielding an estimated wall-clock reduction from 222.5 to 72.0 minutes per episode (3.09× speedup).

5.4.2 Cross-domain Validation. To test whether PoNR captures a protocol-dependent recoverability transition beyond hydrodynamics, we apply CPA to a canonical damped bistable system with bounded intervention and finite horizon H (see Appendix K for the dynamics, protocol $\mathcal{U}$, and the recoverability predicate). Figure 10 contrasts a constrained regime where a recoverability boundary is identifiable, with an over-actuated regime where PoNR is not identifiable in the plotted region.

6 Discussion

Outcome-centric metrics (e.g., success rate, time-to-failure) are lagging for constrained dynamics: they register violations only after recoverability is already lost. PoNR reframes failure analysis as boundary discovery on the snapshot timeline under an audited intervention class and finite horizon. The interval between PoNR



(a) Constrained regime ($u_{\max} = 0.25$) (b) Over-actuated regime ($u_{\max} = 0.5$)

Figure 10: Cross-domain phase-space validation. (a) Empirical recoverability map under the audited protocol $(\mathcal{U}, H)$ (blue: recoverable; pink: unrecoverable/Zombie Zone); dashed contour is the induced boundary. CPA returns a PoNR bracket along the nominal trajectory ($u = 0$): last-recoverable (circle) vs. first-unrecoverable (cross), up to grid/time discretization. (b) Over-actuated regime yields (nearly) uniform recoverability in the plotted region, so CPA finds no PoNR bracket. Definitions and protocol are in Appendix K.

and the first violation constitutes an *actionable window* when it exists at the simulator’s temporal resolution.

CPA operationalizes this idea via snapshot replay and bisection on a recoverability predicate, yielding snapshot-aligned Evidence Objects that support structural–policy decomposition through the policy-induced gap and make counterfactual contrastive mining well-posed.

Two practical concerns are audit cost and the replay assumption. CPA is designed as an offline auditing procedure in a replayable simulator (digital twin), not an online controller; its PoNR labels can be distilled into lightweight monitors for deployment (Sec. 5.3), separating expensive supervision generation from real-time operation. We further evaluate scalability (Sec. 5.4) and validate that PoNR localization remains stable under controlled perturbations to surrogate rollouts (Appendix F.3), which serves as a stress test for error accumulation over the audit horizon. To verify cross-domain behavior, we include a canonical bistable system where dense phase-space auditing is feasible (Appendix K), illustrating PoNR as a protocol-dependent recoverability transition beyond hydrodynamics. Finally, all claims remain conditional on the audited intervention class and observation design; we report a coastline-restricted observation variant as a negative result and discuss implications for localized interventions in Appendix J.

7 Conclusion

We proposed PoNR boundary mining as a leading-boundary view of failure in constrained dynamical environments and introduced CPA to estimate PoNR via snapshot-replay interventions. CPA produces snapshot-aligned Evidence Objects that support causal attribution, structural–policy decomposition, and downstream motif discovery from counterfactual traces.

Experiments on a high-fidelity hydrodynamic control testbed show that PoNR can precede observable violation and provide actionable lead time in risk episodes. Additional analyses validate cross-domain behavior on synthetic dynamics (Appendix K) and robustness under surrogate perturbations (Appendix F.3), supporting CPA as a rigorous tool for protocol-dependent safety boundary discovery.

References

- [1] Mattijs Baert, Pietro Mazzaglia, Sam Leroux, and P. Simoens. 2023. Maximum causal entropy inverse constrained reinforcement learning. *Machine Learning* 114 (2023). doi:10.1007/s10994-024-06653-5
- [2] Thomas Banker and Ali Mesbah. 2025. Model-free Reinforcement Learning for Model-based Control: Towards Safe, Interpretable and Sample-efficient Agents. *ArXiv abs/2507.13491* (2025). doi:10.48550/arxiv.2507.13491
- [3] S. Baron. 2023. Explainable AI and Causal Understanding: Counterfactual Approaches Considered. *Minds and Machines* 33 (2023), 347 – 377. doi:10.1007/s11023-023-09637-x
- [4] Zhenhua Dong, Hong Zhu, Pengxiang Cheng, Xinhua Feng, Guohao Cai, Xiquiang He, Jun Xu, and Ji-Rong Wen. 2020. Counterfactual learning for recommender system. *Proceedings of the 14th ACM Conference on Recommender Systems* (2020). doi:10.1145/3383313.3411552
- [5] Tom Everitt, Marcus Hutter, Ramana Kumar, and Victoria Krakovna. 2021. Reward tampering problems and solutions in reinforcement learning: A causal influence diagram perspective. *Synthese* 198, Suppl 27 (2021), 6435–6467.
- [6] Jaime F Fisac, Anayo K Akametalu, Melanie N Zeilinger, Shahab Kaynama, Jeremy Gillula, and Claire J Tomlin. 2018. A general safety framework for learning-based control in uncertain robotic systems. *IEEE Trans. Automat. Control* 64, 7 (2018), 2737–2752.
- [7] Wensheng Gan, Jerry Chun-Wei Lin, Philippe Fournier-Viger, Han-Chieh Chao, and Philip S Yu. 2019. A survey of parallel sequential pattern mining. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 13, 3 (2019), 1–34.
- [8] Kunal Garg, Songyuan Zhang, Oswin So, Charles Dawson, and Chuchu Fan. 2024. Learning safe control for multi-robot systems: Methods, verification, and open challenges. *Annu. Rev. Control.* 57 (2024), 100948. doi:10.1016/j.arcontrol.2024.100948
- [9] Nathan Grinsztajn, Johan Ferret, Olivier Pietquin, Matthieu Geist, et al. 2021. There is no turning back: A self-supervised approach for reversibility-aware reinforcement learning. *Advances in Neural Information Processing Systems* 34 (2021), 1898–1911.
- [10] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2018. Deep reinforcement learning that matters. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 32.
- [11] Ankita Kushwaha, Kiran Ravish, Preeti Lamba, and Pawan Kumar. 2025. A Survey of Safe Reinforcement Learning and Constrained MDPs: A Technical Survey on Single-Agent and Multi-Agent Safety. *ArXiv abs/2505.17342* (2025). doi:10.48550/arxiv.2505.17342
- [12] Huihan Liu, Shivin Dass, Roberto Martín-Martín, and Yuke Zhu. 2024. Model-based runtime monitoring with interactive imitation learning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 4154–4161.
- [13] Diego Manzananas Lopez, Patrick Musau, Nathaniel P Hamilton, and Taylor T Johnson. 2022. Reachability analysis of a general class of neural ordinary differential equations. In *International Conference on Formal Modeling and Analysis of Timed Systems*. Springer, 258–277.
- [14] Murali Krishna Pasupuleti. 2025. Auditing Black-Box AI Systems Using Counterfactual Explanations. *International Journal of Academic and Industrial Research Innovations(IJAIRI)* (2025). doi:10.62311/nexs/rphcr20
- [15] Matěj Petrlík, Tomáš Báča, Daniel Heřt, Matouš Vrba, Tomáš Krajník, and Martin Saska. 2020. A robust UAV system for operations in a constrained environment. *IEEE Robotics and Automation Letters* 5, 2 (2020), 2169–2176.
- [16] Silviu Pitis, Elliot Creager, and Animesh Garg. 2020. Counterfactual Data Augmentation using Locally Factored Dynamics. *ArXiv abs/2007.02863* (2020).
- [17] Karlo Rado, Mirko Baglioni, and Anahita Jamshidnejad. 2025. Enabling robots to autonomously search dynamic cluttered post-disaster environments. *Scientific Reports* 15, 1 (2025), 34778.
- [18] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J Maddison, and Tatsunori Hashimoto. 2023. Identifying the risks of lm agents with an lm-emulated sandbox. *arXiv preprint arXiv:2309.15817* (2023).
- [19] Stratis Tsirtsis, A. De, and M. Gomez-Rodriguez. 2021. Counterfactual Explanations in Sequential Decision Making Under Uncertainty. (2021), 30127–30139.
- [20] Zefan Wang, Zichuan Liu, Yingying Zhang, Aoxiao Zhong, Lunting Fan, Lingfei Wu, and Qingsong Wen. 2023. RC-Agent: Cloud Root Cause Analysis by Autonomous Agents with Tool-Augmented Large Language Models. *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management* (2023). doi:10.1145/3627673.3680016
- [21] Simon Wiegrefe, Philipp Kopper, R. Sonabend, and Andreas Bender. 2023. Deep learning for survival analysis: a review. *Artificial Intelligence Review* 57 (2023). doi:10.1007/s10462-023-10681-3
- [22] Youxi Wu, Yufei Meng, Yan Li, Lei Guo, Xingquan Zhu, Philippe Fournier-Viger, and Xindong Wu. 2024. COPP-Miner: Top-k Contrast Order-Preserving Pattern Mining for Time Series Classification. *IEEE Transactions on Knowledge and Data Engineering* 36 (2024), 2372–2387. doi:10.1109/tkde.2023.3321749
- [23] Youxi Wu, Yuehua Wang, Yan Li, Xingquan Zhu, and Xindong Wu. 2021. Top-k self-adaptive contrast sequential pattern mining. *IEEE transactions on cybernetics* 52, 11 (2021), 11819–11833.
- [24] Rongwu Xu, Xiaojian Li, Shuo Chen, and Wei Xu. 2025. "Nuclear Deployed!": Analyzing Catastrophic Risks in Decision-making of Autonomous LLM Agents. *ArXiv abs/2502.11355* (2025). doi:10.48550/arxiv.2502.11355
- [25] Qisong Yang, Thiago D Simão, Simon H Tindemans, and Matthijs TJ Spaan. 2023. Safety-constrained reinforcement learning with a distributional safety critic. *Machine Learning* 112, 3 (2023), 859–887.
- [26] Xiang Yin, Bingzhao Gao, and Xiao Yu. 2024. Formal synthesis of controllers for safety-critical autonomous systems: Developments and challenges. *Annual Reviews in Control* 57 (2024), 100940.
- [27] Linrui Zhang, Q. Zhang, Li Shen, Bo Yuan, Xueqian Wang, and Dacheng Tao. 2022. Evaluating Model-free Reinforcement Learning toward Safety-critical Tasks. *ArXiv abs/2212.05727* (2022). doi:10.48550/arxiv.2212.05727
- [28] Qiyuan Zhang, Shu Leng, Xiaoteng Ma, Qihan Liu, Xueqian Wang, Bin Liang, Yu Liu, and Jun Yang. 2024. CVaR-Constrained Policy Optimization for Safe Reinforcement Learning. *IEEE Transactions on Neural Networks and Learning Systems* 36 (2024), 830–841. doi:10.1109/tnnls.2023.3331304

CONTENTS

Abstract	1	F.4	Limitations	17	1103
1 Introduction	1	G	Static Baseline Evaluation Details	17	1104
2 Related Work	2	H	Theoretical Validation on Synthetic Dynamics	17	1105
2.1 From outcome metrics to boundary auditing	2	H.1	System Dynamics	17	1106
2.2 Counterfactual alignment for sequential mining and scalable auditing	2	H.2	Experimental Setup	17	1107
3 Problem Formulation	2	H.3	Results and Phase Space Analysis	17	1108
3.1 Episodes and replayable snapshots	2	I	Implementation Details (Replication Checklist)	17	1109
3.2 Recoverability and point of no return	2	I.1	Tensor Layout and Normalization Pipeline	17	1110
3.3 Structural and policy decomposition	3	I.2	Masking and Stability Clamps	18	1111
4 Methodology	3	I.3	Saved Episode Artifacts and Logging Keys	18	1112
4.1 Evidence Objects: The Mining Unit	3	I.4	Protocol for Capacity Sweep Phase Diagrams	18	1113
4.2 Algorithm: Counterfactual PoNR Auditing (CPA)	3	I.5	Implementation Notes and Sanity Checks	19	1114
4.3 Causal Estimands	3	J	Discarded Variant: Coastline-Based Observation (Negative Result & Failure Boundary)	19	1115
4.4 Identification and Robustness	4	K	Cross-domain Validation on Synthetic Dynamics	19	1117
5 Experiments	4	K.1	System and audit protocol	19	1118
5.1 Auditing Protocol & Experimental Setup	5	K.2	Recoverability predicate and PoNR bracketing	19	1119
5.2 Zombie Zone: Existence and Precedence	5	K.3	Regime contrast	19	1120
5.3 Operational Utility: Regime Taxonomy, Early Warning, and Motif Mining	7				1121
5.4 Scalability and Generalization	7				1122
6 Discussion	8				1123
7 Conclusion	8				1124
References	9				1125
Appendices	10				1126
Contents	10				1127
A Dataset Characteristics	11				1128
A.1 Surrogate Model Fidelity	11				1129
A.2 Dataset Statistics Atlas	11				1130
B Replayable Environment (Hydrodynamic Domain)	11				1131
B.1 State Representation and Surrogate Dynamics	11				1132
B.2 Deterministic Forcing Replay Order	12				1133
B.3 Action Semantics: Drainage and Momentum Damping	12				1134
B.4 Safety Slack, Thresholds, and Termination Horizon	12				1135
B.5 Episode Extraction and Track A/B Taxonomy	12				1136
C CPA Implementation and Evidence Objects	13				1137
C.1 Evidence Object Schema	13				1138
C.2 Snapshot Interface and Time-Travel Replay	13				1139
C.3 Counterfactual PoNR Auditing via Binary Search	13				1140
C.4 Time Indexing Convention (Absolute Time)	13				1141
C.5 PoNR Localization via Binary Search	13				1142
D Early-window Predictability Details	13				1143
D.1 Problem Setup and Leakage Guard	13				1144
E Qualitative Atlas of Failure Modes	14				1145
E.1 Representative Cases (Zombie / Instant / Near-miss)	14				1146
E.2 Extended Analysis of Regime Shift	14				1147
E.3 Additional Regime Shift Example	15				1148
E.4 Contrastive Failure Motif Mining Details	15				1149
F Surrogate Reliability for Counterfactual Auditing	15				1150
F.1 Physics-Informed Hydrodynamic Emulation	15				1151
F.2 Fidelity Against High-fidelity Simulator	16				1152
F.3 Robustness of PoNR Under Noise	16				1153

Notation

Table 3: Summary of Key Notation

Symbol	Description
Core Time & Audit Variables	
t_{fail}	Observed failure time (first constraint violation).
t_{PoNR}	Point of No-Return: last time recoverable under audited capability (conceptual).
t^*	Estimated PoNR time from CPA (used in algorithms and results).
$\hat{t}^*$	Formal estimator of PoNR (output of Algorithm 1).
Δ	Actionable lead time: $\Delta = \max(0, t_{\text{fail}} - t^*)$.
Δ_{gap}	Policy-induced gap: $\Delta_{\text{gap}} = \hat{t}_{\text{oracle}}^* - \hat{t}_{\text{obs}}^*$.
$t_{\text{alarm}}(\tau)$	Alarm time of static threshold τ on slack.
Capacity, Thresholds & Metrics	
D_{max}	Maximum intervention capacity (normalized drainage intensity).
τ	Static alarm threshold on slack (meters).
θ_{safe}	Episode safety threshold (water level).
η	Tolerance parameter for probabilistic recoverability.
H	Audit / rollout horizon (hours).
Late Rate	Percentage of episodes where $t_{\text{alarm}}(\tau) > t_{\text{PoNR}}$.
Alarm Lag	Average delay: $\mathbb{E}[t_{\text{alarm}}(\tau) - t_{\text{PoNR}}]$ (hours).
Framework & Data Components	
x_t, s_t	Replayable system snapshot / physical state at time t .
$y(x_t; i)$	Recoverability indicator (1=recoverable, 0=lost) under intervention i .
$\mathcal{D}$	Dataset of Evidence Objects (EOs).
Track A	Auditable episodes (PoNR localization meaningful).
Track B	Instant-failure episodes ($t_{\text{fail}} < 2$ h).
Zombie Zone	Interval $[t_{\text{PoNR}}, t_{\text{fail}})$: feasible but irrecoverable.
UNO	U-Net Operator surrogate for fast counterfactual rollouts.
General Mathematical Symbols	
$m(s_t)$	Safety slack: $m(s_t) = \theta_{\text{safe}} - \max(H_t)$.
$\mathbb{I}[\cdot]$	Indicator function: $\mathbb{I}[P] = 1$ if P true, else 0.
$\mathbb{E}[\cdot], \mathbb{P}(\cdot)$	Expectation and probability operators.
$\mathcal{U}$	Intervention space (set of admissible control actions).
K	Number of rollout repetitions for stochastic auditing.
α, β	Control parameters for drainage and momentum damping.

A Dataset Characteristics

A.1 Surrogate Model Fidelity

To ensure physical fidelity, surrogate errors are evaluated only over valid water cells, excluding land regions where hydrodynamic states are not meaningful. The valid water mask is extracted directly from environment input channels. We report Mean Absolute Error (MAE) on water cells:

$$\text{MAE} = \mathbb{E}_{(i,j) \in \Omega_{\text{water}}} \left[|H^{\text{gt}}(i, j) - H^{\text{pred}}(i, j)| \right].$$

Figure 11 shows a representative comparison of ground truth (Delft3D) and surrogate water level fields. The observed low water-only MAE of 0.149 m indicates the surrogate reliably captures spatial patterns relevant for constraint evaluation, supporting its use as rollout substrate for CPA.

A.2 Dataset Statistics Atlas

This appendix summarizes the dataset composition and basic statistics of the audited risk episodes, complementing the population-level results in the main text. Our goal is two-fold: (i) to document the coverage and diversity of event intensities and failure horizons, and (ii) to provide transparency on the distribution of key audit outputs (t_{fail} , t^* , and actionable lead time Δ).

Risk Episodes and Taxonomy. A **risk episode** is a replayable window extracted from a longer storm event where the system trajectory approaches the constraint boundary. Each episode is represented as an Evidence Object (Appendix C) and is audited independently. We classify episodes into two primary tracks:

- **Track A:** Episodes where the baseline trajectory fails within the audit horizon, admitting meaningful PoNR localization.
- **Track B:** Near-instant/flash failures with essentially no actionable window. These are reported separately to avoid conflating “inevitable collapse” with “late warning”.

Statistical Overview. Figure 12 provides a detailed statistical atlas of the audited episodes from four perspectives:

- **Top-Left:** Distribution of peak water levels (episode intensity).
- **Top-Right:** Distribution of baseline failure times t_{fail} .
- **Bottom-Left:** Distribution of actionable lead times $\Delta = t_{\text{fail}} - t^*$ for non-censored episodes.
- **Bottom-Right:** Composition of episode types (Track A/B and censoring categories).

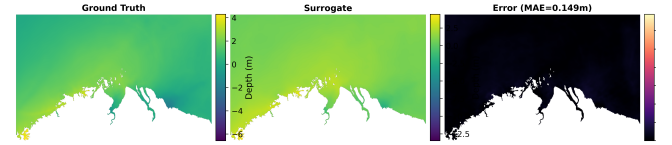


Figure 11: Surrogate Model Fidelity. Ground truth (Delft3D) and surrogate water level fields closely match, with MAE of 0.149 m over valid water cells.

B Replayable Environment (Hydrodynamic Domain)

B.1 State Representation and Surrogate Dynamics

The system state $S_t \in \mathbb{R}^{C \times H \times W}$ represents the hydrodynamic field at time t .

- **Input Tensor:** The model operates on an 8-channel input ($C = 8$). Channels 0–3 encode exogenous forcings (boundary tides, upstream discharge, precipitation, and wind stress); Channels 4–6 encode the hydrodynamic state variables: water level (H), zonal velocity (U), and meridional velocity (V). Channel 7 is reserved for static bathymetry.
- **UNO Surrogate:** We employ a U-Net based Neural Operator (UNO) with a base width of 32. The model is trained to

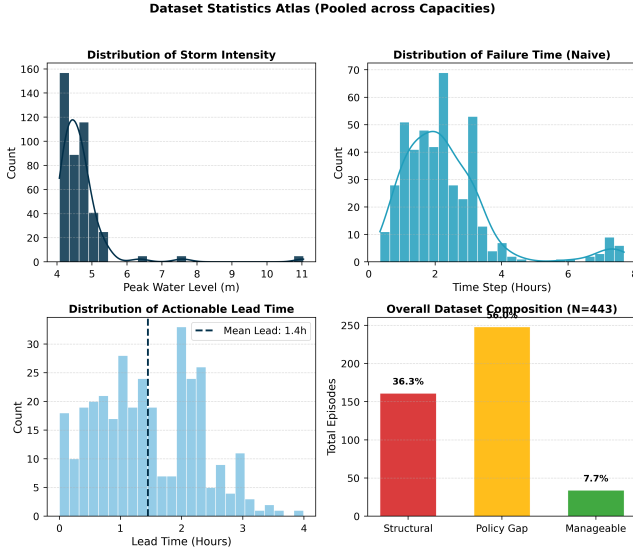


Figure 12: Dataset Statistics Atlas. Comprehensive view of episode intensities, failure times, lead times, and dataset composition.

approximate the transition operator $\mathcal{T} : S_t \mapsto S_{t+1}$ of the shallow water equations.

- **Spatial Masking:** A static binary mask $M \in \{0, 1\}^{H \times W}$ (loaded from `grid_coords.mat`) is applied at every step to zero out non-water cells.

UNO as an increment operator. In implementation, UNO predicts a 3-channel *increment* field $\Delta(H, U, V)$ rather than an absolute next-state. Concretely, after inference we update the endogenous channels by addition: $H_{t+1} \leftarrow H_t + \Delta H_t$, $U_{t+1} \leftarrow U_t + \Delta U_t$, $V_{t+1} \leftarrow V_t + \Delta V_t$. The channel indices are fixed as $\text{IDX_H} = 4$, $\text{IDX_U} = 5$, $\text{IDX_V} = 6$ in the environment.

Land masking. A static mask is loaded from `grid_coords.mat` (e.g., `bGround` or thresholded `Depth`). The environment applies masking after each UNO update by multiplying the full tensor with $(1 - \text{land_mask})$ so that land cells are deterministically zeroed throughout rollouts.

B.2 Deterministic Forcing Replay Order

To ensure a strict counterfactual comparison, we implement a *deterministic forcing replay* mechanism with a fixed within-step order. Let t_{abs} denote the absolute time index in the recorded storm trace, and let the episode be initialized at absolute index t_{start} . The environment maintains a relative counter t (initialized to 0 at reset) such that $t_{\text{abs}} = t_{\text{start}} + t$.

At each simulation step, the execution order is:

- (1) **Action application on endogenous channels:** modify (H, U, V) in-place at the current step.
- (2) **Surrogate dynamics:** run UNO on the full 8-channel tensor and add predicted increments to (H, U, V) .
- (3) **Masking and stability clamp:** apply land masking and clamp (H, U, V) to valid ranges.

- (4) **Forcing overwrite at next absolute time:** increment the relative counter $t \leftarrow t + 1$, compute $t_{\text{abs}}^+ = t_{\text{start}} + t$, and overwrite Channels 0–3 with the recorded forcing slice $\mathcal{F}_{t_{\text{abs}}^+}^{\text{obs}}$ from the dataset tensor.

This *overwrite-after-update* order guarantees that all counterfactual branches share identical exogenous boundary conditions at every absolute time index, and only differ in the intervention applied to endogenous channels.

B.3 Action Semantics: Drainage and Momentum Damping

The intervention action $a_t \in [0, D_{\text{max}}]$ represents a normalized drainage intensity applied *before* the surrogate update. We implement a simple physically-inspired control that (i) removes water mass from the surface elevation field and (ii) damps momentum to emulate flow suppression:

$$H_t \leftarrow H_t - \alpha \cdot a_t, \quad (10)$$

$$U_t \leftarrow U_t \cdot (1 - \beta \cdot a_t), \quad (11)$$

$$V_t \leftarrow V_t \cdot (1 - \beta \cdot a_t). \quad (12)$$

We use $\alpha = 1.0$ and $\beta = 0.2$, consistent with the calibrated environment. Notably, this configuration is intentionally conservative (weakened) in that it induces strong inertia: single-step drainage yields limited effect and requires sustained intervention to alter outcomes.

B.4 Safety Slack, Thresholds, and Termination Horizon

Episode-level threshold. For each extracted episode, we instantiate a fixed safety threshold θ_{safe} based on the episode’s historical peak intensity: $\theta_{\text{safe}} = 0.85 \times \max(H_{\text{historical}})$. In the data generation pipeline, this is implemented by computing the global maximum water-level series, taking its peak, and multiplying by a fixed ratio $\text{THRESHOLD_RATIO} = 0.85$.

Slack and violation. We define the safety slack at time t as $m(S_t) = \theta_{\text{safe}} - \max_{x,y} H_t(x, y)$. A violation is triggered when $m(S_t) < -0.01$, i.e., the maximum water level exceeds the threshold with a small numerical tolerance.

Effective termination horizon. Although the auditor may request longer rollouts, the environment enforces a strict step cap of $T = 24$ (hours) via an internal `max_steps` parameter. Episodes terminate at the first violation or when $t \geq 24$, whichever occurs first. All reported lead times and success rates are computed under this effective 24-step window.

B.5 Episode Extraction and Track A/B Taxonomy

To construct replayable decision episodes that capture the transition into failure (rather than only the aftermath), we employ a risk-aware extraction pipeline with a *scientific backoff* mechanism.

Episode extraction and initialization. We first identify candidate risk windows around constraint-approaching or violating segments (*anchor search*). For each candidate, we sample an episode start

Table 4: Evidence Object fields (summary). This table lists the minimal fields required to reproduce CPA rollouts and PoNR localization.

Field	Description
snapshot_0	Initial state snapshot at episode start
forcing_seq	Recorded forcing schedule indexed by absolute time
t_start	Absolute start time index of the episode window
naive_trace	Slack/trajectory under the naive policy
t_fail	First violation time under naive policy (absolute)
oracle_scan[1..0]	Oracle audit results under max-strength intervention
t_star	Localized PoNR time t^* (absolute), if identifiable
censoring_type	left-/right-censored or observed

time t_{start} from a pre-violation region (e.g., tens of steps before the peak/violation) to ensure the initial snapshot is meaningfully pre-critical.

Scientific backoff. If a sampled start time yields an effectively instant episode (e.g., immediate violation or $t_{\text{PoNR}} \approx t_{\text{start}}$), we iteratively shift t_{start} earlier by a fixed stride and retry for a small number of attempts. This backoff procedure increases the probability that the extracted episode contains a non-trivial pre-PoNR prefix, enabling meaningful lead-time and regime analyses.

Track A/B taxonomy. We partition episodes into two tracks for reporting, as defined in Section 4.4: Track A (Auditable) and Track B (Instant/Flash).

Censoring (visibility of the boundary). Within Track A, we further classify episodes by whether the PoNR transition is observable within the episode window, following the same censoring categories (observed, left-censored, right-censored) defined in Section 4.4.

C CPA Implementation and Evidence Objects

C.1 Evidence Object Schema

Each episode is encapsulated as an *Evidence Object* (EO) that supports deterministic replay. At minimum, an EO contains: (i) an initial snapshot, (ii) a forcing schedule indexed by absolute time, (iii) the naive-policy rollout trace, and (iv) auditing outputs (oracle outcomes and PoNR localization results). We provide a field-level description of EO contents in Table 4.

C.2 Snapshot Interface and Time-Travel Replay

The environment exposes snapshot interfaces (`get_snapshot()` and `load_snapshot()`) to support time-travel interventions. A snapshot captures the full endogenous state, the current absolute time index, and any additional simulator bookkeeping needed for deterministic continuation. Counterfactual rollouts are performed by loading a snapshot at candidate intervention time and executing the oracle policy under the same forcing schedule.

A counterfactual rollout proceeds in discrete hours. At each step, we (i) overwrite forcing channels using the recorded forcing schedule, (ii) apply the intervention to endogenous channels, (iii) advance dynamics using UNO-predicted increments, and (iv) compute slack and check termination. This procedure ensures that counterfactual branches differ only due to interventions, not due to changes in exogenous conditions.

We localize the Point of No Return t^* (denoted as t_{PoNR} in the paper) by binary search over candidate intervention start times along the episode snapshot timeline. The oracle uses a constant maximal action policy, i.e., $a_\tau = D_{\text{max}}$ for all $\tau \geq t$ once the intervention starts. For each queried start time t , we perform a counterfactual rollout under the same forcing schedule and define the recoverability predicate as: the rollout remains feasible if $m(s_\tau) \geq -0.01$ for all τ within the audit horizon. The bisection terminates when the interval between the last *Savable* time and the first *Lost* time is minimized on the discrete 1-hour grid, and we return the *latest* recoverable start time as t^* .

C.3 Counterfactual PoNR Auditing via Binary Search

Algorithm 1 provides the full procedure for localizing the recoverability transition over snapshot indices. In the main text, we describe the audit at a high level and defer implementation details—including censoring handling, rollout-based feasibility estimation, and the bisection loop—to this appendix for completeness and reproducibility.

C.4 Time Indexing Convention (Absolute Time)

We instantiate the safety constraint via a scalar *safety slack* $m(s_t)$ derived from the hydrodynamic field. A state is considered feasible if $m(s_t) \geq -0.01$ and violated if $m(s_t) < -0.01$. An episode terminates upon the first constraint violation or after the fixed rollout horizon of $H = 24$ steps (hours), whichever occurs first. We report all timestamps in absolute time (not re-indexed from t_{start}) to avoid ambiguity in cross-episode alignment.

C.5 PoNR Localization via Binary Search

We localize the structural Point of No Return (PoNR), denoted t_{PoNR} (implementation: t^*), using a bisection procedure over discrete intervention start times.

Oracle capability class. Given an intervention capacity D_{max} , the oracle applies a constant max-action policy $a_\tau = D_{\text{max}}$ for all $\tau \geq t$, starting from a candidate intervention time t .

Recoverability predicate. For a queried start time t , we replay the episode from the snapshot at t under the oracle policy and evaluate feasibility over the audit horizon. We define the binary predicate:

$$\text{Recoverable}(t) = 1 \iff m(s_\tau) \geq -0.01 \quad \forall \tau \in [t, t + H],$$

where $m(\cdot)$ is the safety slack and H is the rollout horizon.

Bisection search. We evaluate $\text{Recoverable}(t)$ on the discrete 1-hour grid within the episode window and perform binary search to find the boundary between recoverable and lost outcomes. The algorithm returns the *latest recoverable* time index (equivalently, the transition point between the last savable step and the first lost step), which we report as t^* .

D Early-window Predictability Details

D.1 Problem Setup and Leakage Guard

We test whether offline CPA labels can be distilled into online early-warning monitors using only *early-window* features.

Algorithm 1 Counterfactual PoNR Auditing on Track-A Episodes (Binary Search over Snapshots)

Input: Track-A episodes with snapshots $\{x_t\}_{t=0}^T$, capability/intervention class ι , horizon H , tolerance η , rollout budget K

Output: PoNR estimate $\hat{t}^*$ (reported as t_{PoNR}), censoring flag(s), Evidence Object dataset $\mathcal{D}$

```

1:  $\mathcal{D} \leftarrow \emptyset$ 
2: for episode  $e$  in Track A do
3:   obtain  $\{x_t\}_{t=0}^T$  and observed  $t_{\text{fail}}$ 
4:   if  $t_{\text{fail}} < 2$  then
5:     continue  $\triangleright$  skip very short episodes under 1-hour resolution
6:   end if
7:    $t_a \leftarrow 0, t_b \leftarrow T$ 
8:   query  $y(x_{t_a}; \iota)$  with  $K$  rollouts; record an EO into  $\mathcal{D}$ 
9:   if  $y(x_{t_a}; \iota) = 0$  then
10:    mark left-censored; continue
11:  end if
12:  query  $y(x_{t_b}; \iota)$  with  $K$  rollouts; record an EO into  $\mathcal{D}$ 
13:  if  $y(x_{t_b}; \iota) = 1$  then
14:    mark right-censored; continue
15:  end if
16:  while  $t_b - t_a > 1$  do
17:     $t_{\text{mid}} \leftarrow \lfloor (t_a + t_b)/2 \rfloor$ 
18:    run  $K$  counterfactual rollouts from  $x_{t_{\text{mid}}}$  under  $do(\iota)$ 
19:    compute  $\hat{p}_{\text{mid}}$  as the fraction feasible on  $[t_{\text{mid}}+1, t_{\text{mid}}+H]$ 
20:     $y_{\text{mid}} \leftarrow \mathbb{I}(\hat{p}_{\text{mid}} \geq 1 - \eta)$ ; record an EO into  $\mathcal{D}$ 
21:    if  $y_{\text{mid}} = 1$  then
22:       $t_a \leftarrow t_{\text{mid}}$ 
23:    else
24:       $t_b \leftarrow t_{\text{mid}}$ 
25:    end if
26:  end while
27:   $\hat{t}^* \leftarrow t_a$ ; record as  $t_{\text{PoNR}}$ 
28: end for
29: return  $\{\hat{t}^*\}, \mathcal{D}$ 

```

Models observe only features computed from a prefix window that strictly precedes the boundary by a margin (e.g., $t \leq t^* - \delta$), preventing information leakage.

We include trend and curvature features such as: (i) first-order slack slope $\Delta \text{slack} / \Delta t$, (ii) second-order curvature $\Delta^2 \text{slack} / \Delta t^2$, (iii) local linear fit residuals and volatility measures.

E Qualitative Atlas of Failure Modes

E.1 Representative Cases (Zombie / Instant / Near-miss)

To demonstrate the diversity of risk scenarios captured by our framework, we present detailed visualizations of three distinct failure modes. These examples illustrate how the counterfactual oracle differentiates between structural inevitability and actionable opportunities.

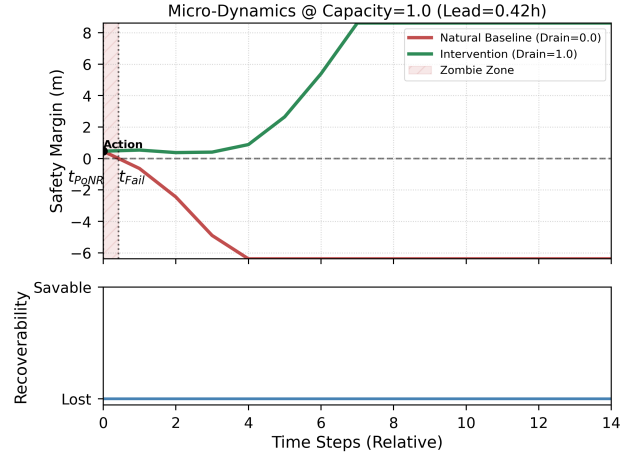


Figure 13: Anatomy of a "Zombie Zone" Episode. Top panel: Safety margin trajectories. Bottom panel: Recoverability status (Blue line). The system enters the irrecoverable state (t_{PoNR} , vertical dotted line) significantly earlier than the observed failure (t_{fail}). During the shaded "Zombie Zone," the operator observes a positive safety margin (Red line > 0) but is structurally doomed, illustrating the danger of reactive thresholds.

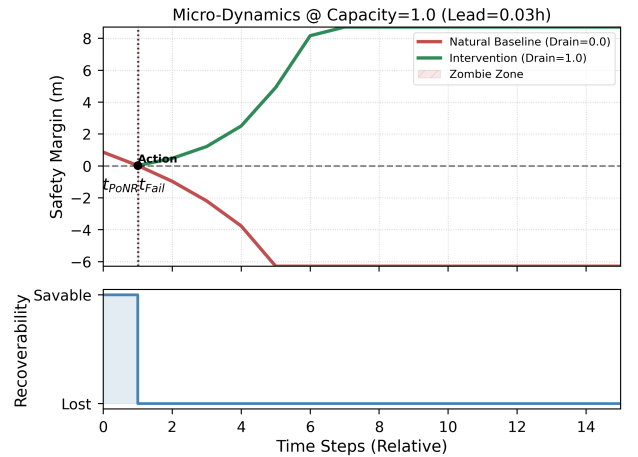


Figure 14: The "Instant Failure" Mode. $t_{\text{PoNR}} \approx t_{\text{start}}$ and $\Delta \approx 0$; no operational window exists.

To complement the population-level statistics ($N = 443$) in Section 5.2.2, we provide detailed visualizations of individual failure modes identified by our CPA framework.

This figure provides a zoomed-in anatomy of the Zombie mode (cf. the Zombie panel in Fig. 13).

E.2 Extended Analysis of Regime Shift

Regime Stratification. We partition failure episodes into three regimes using actionable lead time $\Delta \triangleq \max(0, t_{\text{fail}} - t_{\text{PoNR}})$:

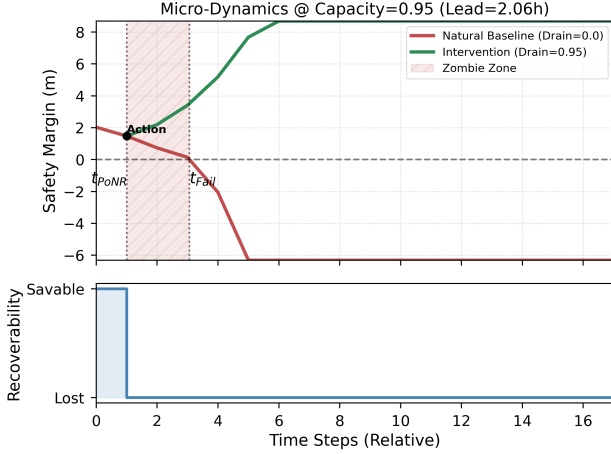


Figure 15: The "Near-Miss" Rescue. The intervention (Green) prevents failure while the baseline (Red) crashes, revealing a narrow recovery path.

- **Structural Regime:** Episodes with a near-zero recovery window ($\Delta < 0.5h$), indicating no meaningful operational window at the simulator’s temporal resolution under the audited intervention class.
- **Policy Gap Regime (Value Zone):** Episodes with an intermediate recovery window ($0.5h \leq \Delta \leq 2.5h$). Recovery is feasible in principle, but the baseline policy fails to exploit the available window.
- **Manageable Regime:** Episodes with a large recovery window ($\Delta > 2.5h$), where ample slack enables robust recovery strategies.

Shift in Failure Dynamics. As $D_{\max}$ increases, Fig. 5 shows failures shifting from *Structural* ($\Delta < 0.5h$) to the *Policy Gap* region. Higher capacity thus converts inevitability into policy-sensitive, windowed failures, where CPA is most informative.

Case Study: The Mechanism of Regime Shift. To physically ground these statistics, Figure 6 presents a controlled counterfactual experiment on a single risk episode. Starting from an identical system state, we simulate the future evolution under two different capacity assumptions. Under restricted capacity ($D_{\max} = 0.80$, Left), the system is structurally dead; no intervention can prevent the crash. However, with enhanced capacity ($D_{\max} = 1.00$, Right), the same scenario becomes *Savable*. This visualizes the "Policy Gap": if a failure occurs in the right-hand scenario, it is not due to physics, but due to the baseline policy’s inability to find the green trajectory. CPA effectively filters out the force majeure cases to focus optimization efforts on the actionable ones.

E.3 Additional Regime Shift Example

E.4 Contrastive Failure Motif Mining Details

We detail the discretization (tokenization) and the contrastive sequential mining operator used to construct the Failure Atlas in Figure 9.

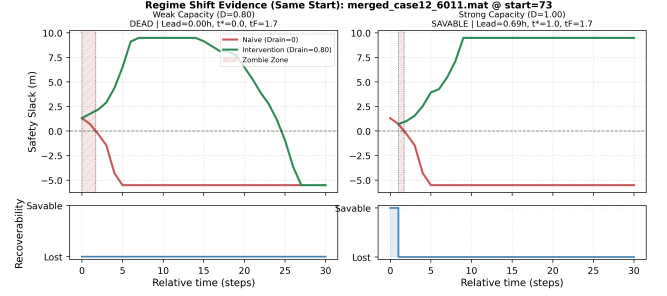


Figure 16: Additional Example of Regime Shift (Complementing Fig. 6). Comparing capacity $D_{\max} = 0.80$ (Left, Structural Failure) vs. $D_{\max} = 1.00$ (Right, Savable) for a different risk episode (Case 12). Consistent with the main text, increased capacity opens a recovery window, transforming the failure from inevitable to actionable.

Episode-level support. We count motif presence per episode (document frequency) rather than raw occurrences to prevent long rollouts from dominating the statistics and to make supports comparable across episodes.

F Surrogate Reliability for Counterfactual Auditing

F.1 Physics-Informed Hydrodynamic Emulation

To accelerate counterfactual auditing in the hydrodynamic domain, we develop a learnable dynamics emulator as a low-cost complement to the high-fidelity simulator.

Architecture. We adopted the **U-shaped Neural Operator (U-NO)** architecture. Unlike standard Convolutional Neural Networks (CNNs) that operate on local pixels, U-NO leverages *Spectral Convolutions* (FFT-based layers) to capture global correlations in the frequency domain. The model follows a U-Net structure with a contracting path (encoder) and an expanding path (decoder) to capture multi-scale hydrodynamic features. The input consists of hydrodynamic state tensors (H, U, V) together with exogenous forcings (e.g., boundary tides, upstream discharge, precipitation, and wind stress), while the output predicts the state residuals at the next time step.

Physics-Informed Training. To ensure physical plausibility, we trained the model using a composite loss function:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \mathcal{L}_{\text{mass}} \quad (13)$$

where $\mathcal{L}_{\text{data}}$ is the relative L_2 error against Delft3D ground truth, and $\mathcal{L}_{\text{mass}}$ is a physics-informed residual loss derived from the continuity equation of shallow water dynamics (checking mass conservation across neighboring grid cells). We utilized an *Automatic Weighted Loss* mechanism to dynamically balance these terms during optimization.

Validation Summary. The model was trained on the Case 0–45 dataset and tested on Case 46–52. Table 5 summarizes state-level prediction fidelity against Delft3D. In particular, the emulator achieves an NSE of **0.9955** for water depth (H), the primary safety variable

Algorithm 2 Contrastive Failure Motif Mining on Snapshot-Aligned EOs (Fisher + BH-FDR)

Input: Target EO rollouts $\mathcal{D}_{\text{tar}}$ (e.g., Panic), reference EO rollouts $\mathcal{D}_{\text{ref}}$ (e.g., Smooth); N -gram length N ; minimum target support τ

Output: Ranked motifs with $(w, \text{OR}(w), p(w), q(w))$

- 1: **Tokenization:** convert each rollout into a token sequence
- 2: **for** rollout ρ in $\mathcal{D}_{\text{tar}} \cup \mathcal{D}_{\text{ref}}$ **do**
- 3: initialize sequence $S(\rho) \leftarrow []$
- 4: **for** $t = 1$ to $T(\rho)$ **do**
- 5: compute slack ratio $r_t \leftarrow (\theta - h_t)/\theta$
- 6: map r_t to context $s_t \in \{\text{Safe}, \text{Warn}, \text{Near}, \text{Crit}\}$
- 7: compute $\Delta u_t \leftarrow u_t - u_{t-1}$ and map to dynamics $a_t \in \{\text{Up++}, \text{Up}, \text{Dn++}, \text{Dn}, \text{St}\}$
- 8: form token $\sigma_t \leftarrow s_t, a_t$ and append to $S(\rho)$
- 9: **end for**
- 10: **end for**
- 11: **Episode-level support (document frequency):** count motif presence per episode
- 12: initialize counters $a(\cdot) \leftarrow 0, b(\cdot) \leftarrow 0$
- 13: **for** rollout ρ in $\mathcal{D}_{\text{tar}}$ **do**
- 14: compute set of unique N -grams $U \leftarrow \text{ngrams}(S(\rho), N)$
- 15: **for** w in U **do**
- 16: $a(w) \leftarrow a(w) + 1$
- 17: **end for**
- 18: **end for**
- 19: **for** rollout ρ in $\mathcal{D}_{\text{ref}}$ **do**
- 20: compute set of unique N -grams $U \leftarrow \text{ngrams}(S(\rho), N)$
- 21: **for** w in U **do**
- 22: $b(w) \leftarrow b(w) + 1$
- 23: **end for**
- 24: **end for**
- 25: **Association testing:** one-sided enrichment in target via Fisher exact test
- 26: initialize list $\mathcal{P} \leftarrow []$ and motif set $\mathcal{W} \leftarrow \{w : a(w) > 0 \text{ or } b(w) > 0\}$
- 27: **for** motif w in $\mathcal{W}$ **do**
- 28: **if** $a(w) < \tau$ **then**
- 29: **continue** ▷ filter low-support motifs in target
- 30: **end if**
- 31: $A \leftarrow a(w), B \leftarrow b(w), C \leftarrow |\mathcal{D}_{\text{tar}}| - A, D \leftarrow |\mathcal{D}_{\text{ref}}| - B$
- 32: compute $(\text{OR}(w), p(w))$ by Fisher exact test on counts (A, B, C, D) (one-sided: enriched in target)
- 33: append $p(w)$ to $\mathcal{P}$
- 34: **end for**
- 35: **Multiple testing control:** apply Benjamini–Hochberg to $\mathcal{P}$ to obtain $q(w)$
- 36: **Ranking:** sort motifs by increasing $q(w)$ and decreasing $\text{OR}(w)$
- 37: **return** ranked motif list

for breach determination. These results support its use for fast rollouts, while boundary validity is additionally supported by fidelity and robustness checks in Appendix F (see also Appendix F.3).

Table 5: Performance metrics of the U-NO hydrodynamic emulator against the high-fidelity simulator (Delft3D) on held-out storms. Water depth (H) is the primary safety variable used in breach and constraint evaluation.

Variable	NSE	RMSE	MAE	Rel L_2	MSE
Water Depth (H)	0.9955	0.0342	0.0156	0.0307	3.78×10^{-3}
Flow Velocity (U)	0.9948	0.0169	0.0078	0.0361	9.54×10^{-4}
Flow Velocity (V)	0.9814	0.0326	0.0142	0.0549	4.92×10^{-3}
Overall	0.9927	0.0296	0.0125	0.0358	3.22×10^{-3}

F.2 Fidelity Against High-fidelity Simulator

CPA requires repeated counterfactual rollouts across many branches; to make large-scale auditing feasible, we rely on a learned hydrodynamic surrogate as the rollout engine. We validate that this surrogate preserves the spatial structure of the critical variable used by the safety slack, by comparing its one-step-ahead water-level prediction against a high-fidelity physics simulator (Delft3D) on matched real cases.

Protocol. For a candidate risk case, we load the corresponding replayable environment and evaluate a representative time step t chosen from the early-to-mid portion of the episode (implemented as $t = \min(10, \lfloor T/2 \rfloor)$). Given the input tensor at time t , we (i) apply the same normalization pipeline as in training, (ii) run the surrogate forward to obtain predicted increments, and (iii) reconstruct the next-step water level field. We then compare this reconstructed field against the ground-truth next-step water level stored in the simulator trace.

F.3 Robustness of PoNR Under Noise

Beyond pointwise fidelity, CPA’s key output is the structural boundary estimate t^* (PoNR). We therefore test whether the estimated t^* is stable under controlled noise perturbations, which serves as a stress test against surrogate imperfections and observation noise.

Noise injection and re-estimation. To assess robustness, we inject multiplicative Gaussian noise into the surrogate outputs during rollouts by perturbing the predicted increments. The PoNR bisection is then re-executed under an identical auditing configuration. We sweep noise levels across $\sigma \in \{0\%, 10\%, 20\%, 30\%, 40\%, 50\%\}$, with multiple independent trials per level to characterize both the mean and variability of the estimated t^* . For each trial, t^* is localized using the same binary-search predicate employed in the main CPA procedure: an intervention initiated at time t is deemed *recoverable* if the oracle rollout maintains feasibility throughout the audit horizon, and we return the latest recoverable t on the discrete 1-hour grid.

Result and interpretation. Figure 17 shows that t^* remains effectively unchanged up to moderate perturbations ($\sigma \leq 30\%$), and only begins to shift (earlier and with larger variance) at high noise levels ($\sigma \geq 40\%$). This pattern matches the expected behavior of a boundary estimator under uncertainty: stronger noise increases rollout variance and makes the recovered boundary more conservative. Importantly, the limited drift over realistic noise ranges supports

that the “Zombie Zone” phenomenon is not a fragile numerical artifact induced by small surrogate errors.

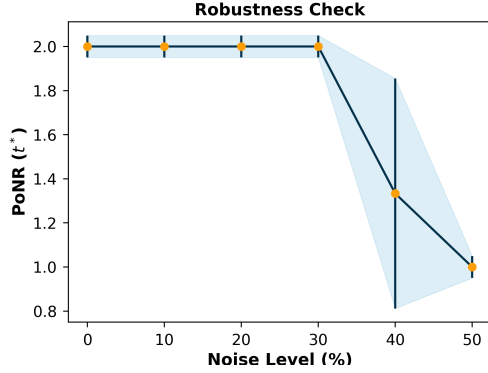


Figure 17: Robustness of PoNR Identification. The identified structural boundary t^* remains stable up to 30% noise intensity. Significant degradation occurs only at high noise levels (> 40%), demonstrating that the Zombie Zone is a robust physical feature rather than a numerical artifact.

F.4 Limitations

The above validity checks are designed as sanity tests that target the failure mode most relevant to CPA: whether surrogate imperfections could spuriously create or erase an apparent PoNR boundary. They do *not* exhaustively characterize all forms of approximation error. In particular, systematic biases (e.g., persistent under-/over-estimation of surge), spatially correlated errors, or distribution shifts under extreme out-of-distribution storms may still move the estimated boundary. Accordingly, we treat surrogate fidelity and robustness as supporting evidence for feasibility and credibility, rather than claiming that PoNR is invariant to all surrogate error modes.

G Static Baseline Evaluation Details

Late-Alarm Phenomenon Analysis. We evaluate a threshold-based detector that triggers an alarm at $t_{\text{alarm}}(\tau) = \min\{t \mid \text{Slack}_t < \tau\}$. We define the *Late-Alarm Rate* as:

$$\text{Rate}_{\text{late}}(\tau) = \mathbb{P}[t_{\text{alarm}}(\tau) > t_{\text{PoNR}}]. \quad (14)$$

We also quantify the *Alarm Lag* as $\mathbb{E}[t_{\text{alarm}}(\tau) - t_{\text{PoNR}}]$ (hours), where larger positive values indicate that the static alarm arrives strictly later than the structural boundary.

Detailed Results and Interpretation. Table 2 shows the comparison across capacities. Even with a conservative threshold ($\tau = 1.0\text{m}$), the static alarm lags behind the structural boundary in **58.2%** of episodes at maximum capacity, indicating that the system is already dynamically irrecoverable before explicit threshold violation. Across capacities, CPA provides an average of **1.4–1.6 hours** of additional reaction time relative to static alarms, consistent with the existence of a non-trivial actionable window measured by Δ .

H Theoretical Validation on Synthetic Dynamics

To demonstrate that the CPA framework captures intrinsic structural boundaries rather than simulation artifacts, we provide a rigorous validation on a controlled synthetic domain.

H.1 System Dynamics

We employ a second-order non-linear bistable system, which serves as a standard proxy for inertia-dominated constrained dynamics (similar to the momentum-driven flow in estuaries). The system is governed by:

$$m\ddot{x} + \gamma\dot{x} - (x - x^3) = u(t), \quad \text{s.t. } |u(t)| \leq u_{\text{max}} \quad (15)$$

where x is the state, $v = \dot{x}$ is velocity, $m = 1.0$ represents inertia, and $\gamma = 0.2$ is the damping coefficient. We set the safety threshold $x_{\text{safe}} = -0.5$ (left potential well) and limited control capacity $u_{\text{max}} = 0.25$.

H.2 Experimental Setup

We generated a failure trajectory starting from $x_0 = 0.1$ with no intervention ($u = 0$). We then applied the CPA algorithm using a bisection search strategy to locate the PoNR t^* . The recoverability predicate $\mathcal{R}(s, H)$ was evaluated by rolling out the dynamics for $H = 15\text{s}$ under the optimal recovery policy $\pi_{\text{rec}}(x) = -u_{\text{max}} \cdot \text{sgn}(\dot{x})$.

H.3 Results and Phase Space Analysis

Figure 11 presents the phase portrait of the validation.

- **Ground Truth Map:** The background regions were generated via exhaustive grid auditing (100×100 resolution). The pink region denotes the *Zombie Zone* $\mathcal{Z}$, while the blue region denotes the *Recoverable Basin* $\mathcal{S}_{\text{rec}}$.
- **Alignment with Separatrix:** The boundary between the blue and pink regions corresponds to the theoretical **separatrix** of the effective potential energy surface.
- **Precision:** The PoNR identified by CPA (red dot) aligns perfectly with the intersection of the failure trajectory and the separatrix.

This experiment confirms that CPA is domain-agnostic and capable of identifying precise topological boundaries in continuous dynamical systems.

I Implementation Details (Replication Checklist)

Appendix B specifies the *conceptual* replayable environment (state channels, forcing replay order, action semantics, and slack / termination). This section complements it with *code-level replication details* extracted from our implementation: (i) tensor layout and normalization/denormalization, (ii) masking and clamping, (iii) the exact artifact schema and logging keys saved per episode.

I.1 Tensor Layout and Normalization Pipeline

Channel convention and layout. The environment state is stored as a 3D tensor with 8 channels, consistent with Appendix B.1. Internally, tensors are kept in CHW layout (PyTorch convention) and

cast to float32. The endogenous channels are indexed as:

$$\text{IDX_H} = 4, \quad \text{IDX_U} = 5, \quad \text{IDX_V} = 6,$$

and the exogenous forcing channels occupy indices 0–3.

UNO predicts increments. The surrogate model predicts *increments* for (H, U, V) rather than absolute next states:

$$(H, U, V)_{t+1} = (H, U, V)_t + (\Delta H, \Delta U, \Delta V)_t.$$

In implementation, the UNO forward pass returns a 3-channel increment field that is added in-place to $\text{IDX_H}/\text{IDX_U}/\text{IDX_V}$.

Normalization/denormalization constants. For faithful replication, it is critical to mirror the exact scaling used at inference. We apply per-channel max-based normalization on the 8-channel input and a corresponding rescaling on the 3-channel output increments. Concretely, we define a vector of input scales

$$\text{IN_MAX_8} \in \mathbb{R}^8$$

and output scales

$$\text{OUT_MAX_3} \in \mathbb{R}^3,$$

and implement:

$$S_t^{\text{norm}}[c] = \frac{S_t[c]}{\text{IN_MAX_8}[c]}, \quad \Delta^{\text{phys}} = \Delta^{\text{norm}} \odot \text{OUT_MAX_3}.$$

In our released code, these constants are instantiated as:

$$\text{IN_MAX_8} = \{0.5, 0.1, 0.2, 0.2, 5.0, 0.7, 0.7, 1.0\},$$

$$\text{OUT_MAX_3} = \{0.03, 0.01, 0.01\},$$

where the first four correspond to forcing channels, the next three correspond to (H, U, V) magnitudes, and the last corresponds to bathymetry/geometry scaling. This normalization choice is intentionally lightweight (affine-free) and was found sufficient for stable surrogate inference in our one-step residual prediction setting.

Note on forcing replay vs. normalization. Normalization is applied only for UNO inference. The forcing replay itself is done in the physical state tensor: after each step we overwrite Channels 0–3 with the recorded forcing slice (Appendix B.2), and the resulting tensor is then normalized *only when* fed into UNO. This separation ensures (i) deterministic forcing alignment in physical units and (ii) a consistent input distribution for the surrogate.

I.2 Masking and Stability Clamps

Land mask extraction. We load a static land/water mask from `grid_coords.mat`. In the coastal implementation, the code constructs a boolean `land_mask` (1 = land, 0 = water), and also logs water/land coverage ratios for sanity checks (e.g., `water_ratio`, `land_ratio`, `coast_ratio`). All masking operations are performed deterministically.

Mask application. After each UNO update, we zero out all channels at land cells:

$$S_t[:, i, j] \leftarrow 0 \quad \text{if} \quad \text{land_mask}[i, j] = 1.$$

We apply the mask in-place using broadcasting (multiplying by $(1 - \text{land_mask})$) to avoid creating nondeterministic NaN patterns during rollouts.

Hard clamping for numerical stability. To bound surrogate drift and ensure physically plausible replay, we clamp only the endogenous channels after the increment update:

$$H \in [-5, 10], \quad U, V \in [-5, 5],$$

implemented as in-place clipping on the $\text{IDX_H}/\text{IDX_U}/\text{IDX_V}$ channels. This clamp is applied *before* forcing overwrite at the next time step, consistent with the within-step order in Appendix B.2.

I.3 Saved Episode Artifacts and Logging Keys

We store the full experimental corpus as a Python pickle (e.g., `full_experiment_data_coast.pkl`) containing a dictionary for each episode. Each episode dictionary is designed to be self-contained enough for: (i) plotting case atlases (Appendix E), (ii) producing population statistics (Sec. 5.2.2).

Episode-level metadata. Each episode saves:

- `case_id`: a unique identifier (e.g., `merged_case11_6010.mat_s95`) encoding the storm case and snapshot index;
- `t_start_abs`: the absolute start time index of the episode window in the underlying storm trace;
- `peak_coast`, `thr`, `thr_ratio`: episode intensity proxy and the instantiated episode-level threshold θ_{safe} ;
- `naive_trace`: baseline rollout trace (time series) including at least slack trajectory and termination info;
- `t_fail_abs`: the first violation time under baseline, in absolute time;
- `track`: "A" or "B" (Appendix B.5);
- `censoring`: categorical label (observed / left-censored / right-censored).

Oracle scan payloads (per capacity). For each intervention capacity D_{max} , we store a structured record under a formatted key:

`oracle_scan["D0.80"], oracle_scan["D0.85"], ...`

where the key is generated by `fmt_key(d)  $\mapsto$  "D{d:.2f}"` with `'.'` removed (e.g., `0.80  $\mapsto$  D0.80`). Each scan payload includes:

- `t_star_abs`: localized PoNR estimate t^* (absolute time);
- `lead`: actionable lead time $\Delta = t_{\text{fail}} - t^*$ in hours/steps (for observed episodes);
- `bracket`: whether a valid *Savable/Lost* bracket was found for bisection;
- `details`: optional debugging fields (e.g., last savable index, first lost index, intermediate predicate results).

This design makes the capacity sweep *replay-free at plotting time*: all t^* points needed for phase diagrams can be read directly from the pickle without re-running the oracle.

I.4 Protocol for Capacity Sweep Phase Diagrams

Sweep procedure. For each extracted episode, we run the same CPA pipeline while varying only the oracle capacity:

- (1) Load the episode environment initialized at `t_start_abs`.
- (2) Run the baseline rollout to obtain the `naive_trace` and the `t_fail_abs`.
- (3) For each D_{max} in the sweep set, run PoNR localization by bisection (Appendix C.5) using an oracle that applies a constant action $a_\tau = D_{\text{max}}$ for all $\tau \geq t$.

(4) Save $\{t^*, \Delta\}$ into `oracle_scan[fmt_key(D_max)]`.

All other controls are held fixed: forcing replay, masking/clamping, slack threshold, and the effective horizon (`max_steps`).

Plotting convention. Each phase diagram plots (t_{fail}, t^*) for the corresponding capacity, using only episodes that yield a valid bisection bracket (`bracket=True`). Points below the diagonal $t^* = t_{\text{fail}}$ correspond to “Zombie Zone” episodes ($t^* < t_{\text{fail}}$), while points near the diagonal indicate near-instant loss of recoverability. The key qualitative claim is that the below-diagonal mass persists across capacities, i.e., PoNR is not a single-capacity artifact but a stable leading-boundary signal across feasible oracle strengths.

Why this is not redundant with Appendix B. Appendix B defines *what* the environment does conceptually. This section defines *how* the sweep is executed and *what* is saved so that the plots can be reproduced exactly from stored artifacts (including key names and filtering rules). In particular, the sweep requires that (i) t is tracked in absolute time, (ii) forcing overwrite is done at the correct absolute index after each step, and (iii) the oracle scan results are persisted per capacity with a stable key format; these are replication-critical but would be distracting in the conceptual environment definition.

I.5 Implementation Notes and Sanity Checks

Absolute-time bookkeeping. The environment maintains a relative counter $t \in \{0, \dots\}$ for stepping, but all logged timestamps are converted to absolute time via:

$$t_{\text{abs}} = t_{\text{start_abs}} + t.$$

We store `t_start_abs`, `t_fail_abs`, and `t_star_abs` to prevent silent off-by-`t_start` errors when aggregating across episodes.

Effective horizon and termination. While some scripts may request longer audit horizons, the hydrodynamic environment enforces `max_steps=24` as the effective termination cap. All phase-diagram and lead-time results are computed under this same cap, ensuring consistent comparability across capacities.

Determinism. Given a fixed initial snapshot, fixed forcing schedule, and fixed intervention schedule, the surrogate-driven rollout is deterministic: the only source of variability in robustness experiments (Appendix F.3) is the explicitly injected noise into surrogate outputs. This property is what allows Evidence Objects (Appendix C.1) to function as replayable audit artifacts.

J Discarded Variant: Coastline-Based Observation (Negative Result & Failure Boundary)

We replace the primary observable from a global all-water peak statistic to a coastline-restricted extreme (or coastline summary), while keeping the intervention unchanged: a uniform drainage action applied to the entire domain. The CPA procedure (snapshot-aligned replay and boundary search) is otherwise identical.

Under this coastline metric, boundary localization becomes unstable and/or yields degenerate actionable margins: many episodes exhibit near-zero recoverability window under the strongest available global intervention, and the estimated boundary can vary

noticeably with minor implementation choices (e.g., thresholding, coastline mask details, or mesh resolution).

This behavior is not merely “harder coastal physics,” but a conundrum at the definition level for our goal. The coastline observable acts as a localized extreme, which is influenced by small-scale geometry, boundary reflections, and numerical noise, while the intervention remains global and spatially uniform. Consequently, the estimated PoNR no longer cleanly reflects a recoverability boundary of the environment–policy system; instead it becomes a composite of (i) true dynamics, (ii) metric-specific local amplification, and (iii) simulator artifacts. This violates our desideratum that PoNR be comparable across settings under a fixed auditing definition.

CPA/PoNR is best validated under matched observation and intervention structure. When a local-extreme observation is paired with a global uniform action model, recoverability becomes strongly entangled with operational feasibility and simulator-specific noise; in this regime, a coastline metric is not an appropriate primary benchmark for isolating the PoNR concept. This negative result motivates future extensions that introduce spatially localized interventions (e.g., region-specific pumping or barrier actions) or multi-observer auditing to restore controllability–observability symmetry.

K Cross-domain Validation on Synthetic Dynamics

K.1 System and audit protocol

We consider a damped bistable system:

$$\dot{x} = v, \quad \dot{v} = \frac{x - x^3 - \gamma v + u}{m}, \quad |u| \leq u_{\text{max}}, \quad (16)$$

with safe set $\mathcal{S} = \{(x, v) : x \leq x_{\text{safe}}\}$ and audit horizon H . Unless otherwise stated, the audited intervention class is a fixed maximal recovery protocol $\mathcal{U} = \{u(t) \equiv -u_{\text{max}}\}$.

K.2 Recoverability predicate and PoNR bracketing

Given $(\mathcal{U}, H)$, we define the recoverability predicate

$$R_{\mathcal{U}, H}(s_0) = \mathbb{I} \left[\min_{t \in [0, H]} x(t; s_0, u) \leq x_{\text{safe}} \text{ under } u(\cdot) \in \mathcal{U} \right]. \quad (17)$$

For visualization, we compute an *empirical* phase-space recoverability map by brute-force evaluation of $R_{\mathcal{U}, H}(x, v)$ on a grid. Independently, CPA applies bisection to $R_{\mathcal{U}, H}(s_t)$ along an observed trajectory $\{s_t\}$ and returns a discrete bracket: the last recoverable snapshot t_{safe} and the first unrecoverable snapshot t_{fail} , so $t^* \in (t_{\text{safe}}, t_{\text{fail}}]$ up to discretization.

K.3 Regime contrast

Figure 10a (main text) shows a constrained regime where a protocol-induced recoverability boundary separates recoverable states from the Zombie Zone. Figure 10b provides a sanity check: when control authority is sufficiently large (or H is sufficiently long), the recoverability boundary can move outside the plotted region, making PoNR non-identifiable on the chosen trajectory; CPA correspondingly reports no PoNR bracket in this regime.